ДИСЦИПЛИНА «Разработка программных средств поиска и анализа киберугроз»

ИНСТИТУТ «Институт кибербезопасности и цифровых технологий»

КАФЕДРА КБ-2 «Информационно-аналитические системы кибербезопасности»

ВИД УЧЕБНОГО МАТЕРИАЛА Лекция

ПРЕПОДАВАТЕЛЬ Новиков Евгений Иванович

СЕМЕСТР 9 семестр 2026-2027 учебный год

ЛЕКЦИЯ 8. СТАТИСТИЧЕСКИЕ МЕТОДЫ ОБНАРУЖЕНИЯ АНОМАЛИЙ В КИБЕРБЕЗОПАСНОСТИ

План лекции:

1. Понятие и виды аномалий в кибербезопасности.
2. Постановки задачи обнаружения аномалий.
3. Обзор методов обнаружения аномалий.
4. Статистические методы выявления аномалий.
5. Практические рекомендации по применению статистических методов.

Статистические методы обнаружения аномалий в кибербезопасности

Современные корпоративные системы генерируют миллиарды событий в сутки: сетевые пакеты, записи журналов аутентификации, обращения к файловой системе, вызовы системных API.

Традиционные сигнатурные средства защиты (антивирусы, IDS на правилах) способны обнаружить лишь известные угрозы. Однако целевые атаки (APT), инсайдерские угрозы, атаки нулевого дня по определению не имеют заранее известной сигнатуры.

Именно здесь на первый план выходит парадигма обнаружения аномалий (anomaly detection): вместо того чтобы искать «известное плохое», мы моделируем «нормальное» поведение системы и фиксируем существенные отклонения от него.

Эта парадигма реализована в NTA/NDR-системах, UEBA-платформах, поведенческих модулях EDR-агентов и является одной из наиболее активно развивающихся областей на стыке машинного обучения и информационной безопасности.

1. Понятие и виды аномалий в кибербезопасности

Основные термины и определения

Наблюдение — единица анализа: отдельное соединение, сеанс, событие входа либо агрегат за фиксированный интервал. Важно определить единицу до выбора метода. Пять ошибок входа за минуту и пять ошибок за сутки описывают разные ситуации.

Признак — измеряемая характеристика наблюдения: число ошибок входа, объём переданных данных, средняя продолжительность сеансов. Набор значений признаков одного наблюдения образует вектор. Если анализируется один признак, модель одномерная; если несколько — многомерная.

Модель нормы — описание ожидаемого поведения определённого объекта при определённых условиях. Это может быть среднее с мерой разброса, центральный диапазон значений, вероятностное распределение либо совместная структура нескольких признаков. Модель всегда имеет область применимости: например, один сервер, одинаковая длительность окна и сопоставимый режим работы.

Профиль нормы — практическое название параметров такого описания для конкретной группы или объекта. Профиль сервера резервного копирования нельзя без проверки переносить на рабочую станцию сотрудника.

Разметка атаки и признаки решают разные задачи. Признаки подаются детектору. Разметка сообщает проверяющему, что на самом деле происходило; она может быть получена в результате расследования инцидентов.

Выброс — это наблюдение, значение которого заметно удалено от основной массы данных по статистическому критерию (находится за пределами доверительного интервала или распределения). Выброс определяется прежде всего на основании статистических свойств выборки.

Например, обычный объём сетевого соединения составляет от 1 до 20 МБ, а одно соединение содержит 900 МБ данных. С точки зрения распределения значений это выброс.

Причинами появления выброса могут быть:

- ошибка измерения;
- неисправность оборудования;
- ошибка регистрации события;
- неправильная единица измерения;
- редкий, но законный процесс;
- вредоносная активность.

Следовательно, выброс не обязательно представляет угрозу и не всегда содержит полезную информацию.

Аномалия — отклонение от ожидаемого поведения объекта или процесса относительно принятой модели нормы. Оно может проявляться в величине, сочетании признаков или последовательности событий.

В литературе слова «выброс» и «аномалия» часто используются как синонимы; универсального строгого разделения нет. В этой лекции статистическое срабатывание рассматривается как указание на потенциальную аномалию.

Важное отличие: аномалия контекстно зависима. То, что выглядит как аномалия в одном сценарии, в другом может быть нормой.

Например:

- передача 500 МБ данных сервером резервного копирования может быть нормальной;
- передача 500 МБ с компьютера бухгалтера на неизвестный внешний адрес может быть аномальной;
- вход администратора в систему в 03:00 может быть редким, но допустимым во время плановых работ;
- вход обычного сотрудника в 03:00 с нового устройства может потребовать расследования.

Таким образом, одно и то же числовое значение может быть как нормальным, так и аномальным в зависимости от пользователя, времени, устройства и выполняемой

операции.

Можно сказать так: любой выброс — это аномалия, но не каждая аномалия — выброс.

Сравнение понятий.

Критерий	Выброс (outlier)	Аномалия (anomaly)
Природа отклонения	Статистическое: значение лежит за пределами доверительного интервала или распределения	Семантическое: поведение противоречит модели нормального функционирования системы
Контекст	Не требуется; определяется исключительно числовой метрикой	Контекст обязателен (время, роль пользователя, сегмент сети)
Причина	Может быть артефактом измерения, ошибкой ввода, редким, но легитимным событием	Предположительно порождена другим процессом (атака, сбой)
Действие	Часто исключается из выборки при предобработке	Является целевым объектом исследования

Тревога — результат принятого правила: детектор отметил наблюдение для проверки. Тревога может оказаться полезной или ложной относительно поставленной задачи.

Атака — вредоносное действие или совокупность действий, установленные по содержательным основаниям (результатам расследования). Статистическая необычность сама по себе атаку не доказывает. Она также может быть связана со сбоем, изменением нагрузки, ошибкой данных или законной редкой операцией.

Общую идею обнаружения аномалий можно представить следующим образом:

$$\text{Оценка аномальности} = \text{степень отклонения наблюдения от нормального пов}$$

Чем сильнее наблюдение отличается от изученного профиля, тем выше его оценка аномальности — anomaly score.

Для принятия решения об отнесении объекта к аномальным выбирается порог:

$$\hat{y}(x) = \begin{cases} \text{аномалия}, & s(x) > T, \\ \text{норма}, & s(x) \le T, \end{cases}$$

где:

- x — анализируемое наблюдение;
- $s(x)$ — числовая оценка аномальности наблюдения;
- T — установленный порог.

Оценка и порог выполняют разные функции. Модель определяет, насколько объект отличается от опорных данных. Порог определяет, какие из отличий будут отмечены. Изменить порог не означает заново обучить модель, если её параметры остаются прежними.

Виды аномалий

Обычно выделяют три основных вида аномалий:

- точечные;
- контекстуальные;
- коллективные.

Точечная аномалия — отдельное наблюдение, существенно отличающееся от остальных данных.

Примеры:

- соединение с исключительно большим объёмом трафика;
- процесс, использующий аномально большой объём оперативной памяти;
- единичный вход из страны, с которой пользователь никогда не работал;
- запрос к необычному сетевому порту;
- запуск неизвестного исполняемого файла.

Предположим, за один сеанс пользователи обычно скачивают от 1 до 30 файлов. Если за один сеанс было скачано 10 000 файлов, это может рассматриваться как точечная аномалия.

Точечные аномалии сравнительно легко обнаруживать статистическими методами, которые анализируют отклонения от выбранной модели нормы. Эти отклонения рассматриваются как потенциальные аномалии. Затем дополнительная проверка устанавливает, объясняются ли они штатной деятельностью или связаны с нарушением, сбоем либо атакой.

Контекстуальная аномалия — наблюдение, которое считается аномальным только при определённых условиях.

Само значение может быть допустимым, но становится подозрительным с учётом контекста.

Контекстными признаками могут быть:

- время;
- день недели;
- местоположение;
- роль пользователя;
- тип устройства;

- сегмент сети;
- этап бизнес-процесса;
- предшествующие события.

Примеры:

- вход в систему в 14:00 является нормальным, а такой же вход в 03:00 — подозрительным;
- использование PowerShell администратором может быть нормальным, а офисным сотрудником — нетипичным;
- передача большого объёма данных сервером резервного копирования является нормальной, а пользовательским компьютером — аномальной;
- подключение из какой-либо страны может быть нормальным для одного сотрудника и необычным для другого.

Для обнаружения контекстуальных аномалий недостаточно анализировать один признак. Необходимо учитывать взаимосвязь нескольких характеристик события - применять корреляционный анализ.

Коллективная аномалия — группа наблюдений, которая в совокупности образует необычное поведение, хотя каждое отдельное событие может выглядеть нормально.

Примеры:

- медленное сканирование портов;
- подбор пароля с большими интервалами между попытками;
- периодические обращения вредоносной программы к командному серверу;
- постепенная передача данных небольшими порциями;
- последовательность «вход → повышение привилегий → доступ к критичному серверу → выгрузка данных»;
- большое число обычных запросов от множества устройств во время распределённой DDoS-атаки.

Коллективные аномалии особенно важны в кибербезопасности. Современные атаки часто специально разбиваются на небольшие действия, чтобы каждое из них не превышало установленные пороги.

Помимо рассмотренных видов аномалий выделяют глобальные и локальные аномалии.

Глобальная аномалия значительно отличается от всего набора данных.

Например, сетевое соединение длительностью 20 часов, если почти все остальные соединения продолжаются несколько минут.

Локальная аномалия отличается только от близких или похожих наблюдений.

Например, объём трафика 500 МБ может быть нормальным для серверов, но аномальным среди рабочих станций отдела кадров.

Локальные аномалии часто сложнее глобальных, поскольку требуют сравнения объекта с его окружением, а не со всей выборкой.

2. Режимы обучения и место статистических методов

Выбор метода обнаружения аномалий зависит от того, какие данные доступны для обучения.

Обучение только на нормальных данных.

В распоряжении исследователя имеются данные, которые считаются подтвержденно нормальными. Модель обучается описывать границы нормального поведения. Всё, что выходит за эти границы, рассматривается как аномалия.

Такая постановка задачи связана с **обнаружением новизны** (novelty detection) и моделированием одного класса. «Новизна» означает отклонение от изученного распределения, а не обязательное отсутствие похожего объекта во всей истории организации.

Обучающая выборка должна быть достаточно чистой. Если в ней присутствуют незамеченные атаки, модель может принять их за нормальное поведение. Нормальная история должна быть не только чистой, но и представительной. Если в ней нет штатной ночной нагрузки или резервного копирования, их появление может вызвать ложные тревоги.

Подход удобен для кибербезопасности, поскольку собрать нормальные данные часто легче, чем получить примеры всех возможных атак.

Обучение без учителя.

Алгоритм получает значения признаков без сведений об атаках. История может быть загрязнена: содержать невыявленные атаки и другие отклонения. Обычно предполагают, что основная часть данных отражает штатное поведение и аномалии не определяют всю структуру выборки.

Здесь могут использоваться те же методы, что при обучении модели только на норме. Неразмеченная выборка может использоваться как для поиска необычных объектов внутри неё, так и для обучения модели, применяемой к будущим событиям. Для проверки будущего мониторинга нужен отдельный проверочный интервал в обоих случаях.

Главный недостаток — алгоритм обнаруживает необычное поведение, но не знает, является ли оно атакой.

Обучение с учителем.

Имеются размеченные примеры двух или нескольких классов:

- нормальное событие;
- атака;
- иногда — конкретные типы атак.

Тогда задача обнаружения аномалий превращается в обычную классификацию.

Преимущество — возможность напрямую обучить модель распознавать известные атаки.

Недостатки:

- требуется большой размеченный датасет;
- атаки встречаются редко - классы обычно сильно несбалансированы;
- новые виды атак могут отсутствовать в обучающей выборке.

Строго говоря, такая постановка чаще называется обнаружением атак, а не классическим поиском аномалий.

Обучение с частичной разметкой.

Имеется небольшое количество размеченных данных и большой объём неразмеченных событий.

Это соответствует частично контролируемому или полуконтролируемому обучению.

Такой вариант реалистичен для SOC: аналитики могут проверить и разметить только небольшую долю событий, поступающих от средств мониторинга.

3. Обзор методов обнаружения аномалий

Существует достаточно большое количество методов выявления аномалий, которые целесообразно классифицировать по используемой парадигме обучения и типу модели.

1. Методы на основе правил и порогов.

Событие признаётся аномальным, если выполняется заданное условие.

Примеры правил:

- более 10 неудачных входов за 5 минут;
- передача более 1 ГБ данных на внешний адрес;
- запуск PowerShell офисным пользователем;
- подключение к запрещённому порту;
- вход с двух удалённых мест за физически невозможный промежуток времени.

Преимущества:

- простота реализации;
- понятность результата;
- высокая скорость работы;
- возможность учитывать экспертные знания.

Ограничения:

- правила необходимо создавать и обновлять вручную;
- жёсткие пороги не учитывают индивидуальное поведение;
- появляются ложные срабатывания;
- неизвестные сценарии атак могут быть пропущены.

Правила хорошо обнаруживают известные нарушения, но хуже работают с новыми и изменяющимися атаками.

2. Статистические методы.

Основаны на построении модели распределения нормальных значений признаков и выявлении наблюдения с низкой вероятностью появления.

Методы:

- Z-оценка;
- межквартильный размах, IQR;
- медианное абсолютное отклонение, MAD;
- квантильные пороги;
- оценка вероятностного распределения;
- критерий Граббса;
- расстояние Махаланобиса;
- контрольные карты.

Преимущества:

- простота;
- высокая скорость;
- интерпретируемость.

Ограничения:

- некоторые методы предполагают определённое распределение данных;
- чувствительность к самим выбросам;
- сложности при работе с большим числом признаков;
- слабое обнаружение сложных контекстуальных аномалий.

3. Методы оценки плотности.

Эти методы оценивают, насколько плотно расположены объекты вокруг конкретного наблюдения. Объект считается аномальным, если плотность вокруг

него значительно ниже, чем у соседей.

Методы:

- Histogram-Based Outlier Score (HBOS);
- Kernel Density Estimation (KDE);
- Gaussian Mixture Model (GMM).

Преимущества:

- обнаружение локальных аномалий;
- учёт структуры окружения объекта;
- отсутствие необходимости заранее задавать форму кластеров.

Ограничения:

- чувствительность к числу соседей;
- необходимость масштабирования;
- высокая стоимость обработки больших выборок;
- сложность интерпретации итоговой оценки.

4. Методы на основе построения границы нормального поведения.

Основная идея этой группы — построить в пространстве признаков область, соответствующую нормальным наблюдениям. Объекты за её границей рассматриваются как потенциальные аномалии. При этом модель может допускать, что небольшая часть обучающих наблюдений окажется вне нормальной области.

Наиболее распространенным методои этой группы является One-Class SVM.

Преимущества:

- возможность строить нелинейные границы;
- эффективность на выборках умеренного размера;
- пригодность для обучения на нормальных данных.

Ограничения:

- высокая чувствительность к масштабированию;
- сложность выбора параметров;
- сравнительно низкая скорость на больших выборках;
- чувствительность к загрязнению обучающих данных аномалиями.

5. Методы на основе расстояний.

Основная идея: аномальные объекты находятся далеко от большинства наблюдений.

Для каждого объекта может вычисляться расстояние до ближайших соседей. Если оно велико, объект получает высокую оценку аномальности.

Наиболее распространенные методы:

- k ближайших соседей (kNN);
- Local Outlier Factor (LOF).

Преимущества:

- простая и понятная идея;
- отсутствие предположений о распределении данных;
- возможность обнаружения многомерных аномалий.

Ограничения:

- необходимость масштабирования признаков;
- высокая вычислительная стоимость на больших данных;
- чувствительность к выбору метрики расстояния;
- снижение эффективности в пространствах высокой размерности.

6. Методы кластеризации.

Кластеризация разделяет данные на группы похожих объектов.

Аномальными могут считаться:

- объекты, не попавшие ни в один кластер;
- объекты, расположенные далеко от центра кластера;
- объекты из очень маленьких кластеров.

Применяемые основные алгоритмы:

- k-means;
- DBSCAN;
- HDBSCAN;
- иерархическая кластеризация.

Например, сетевая активность может образовать кластеры:

- обычный веб-трафик;
- почтовый трафик;
- обращения к файловым серверам;
- небольшая группа необычных соединений.

Преимущества:

- отсутствие необходимости в разметке;
- возможность выявлять структуру данных;
- удобство для исследовательского анализа.

Ограничения:

- небольшой кластер не обязательно является аномалией;
- аномалия может попасть в крупный кластер нормальных событий;

- результат зависит от параметров алгоритма.

7. Методы на основе изоляции.

Основная идея этой группы: аномальное наблюдение обычно удаётся отделить от остальных объектов меньшим числом случайных разбиений пространства признаков. Например, если большинство соединений передаёт от 1 до 20 МБ, а одно — 900 МБ, случайное разделение по объёму трафика часто позволяет быстро изолировать такое соединение. Для отделения одного типичного соединения от множества похожих обычно требуется больше разбиений.

В этой группе наиболее часто применяется метод Isolation Forest, который строит множество случайных деревьев изоляции. По средней длине пути до изоляции объекта рассчитывается оценка аномальности: чем короче путь, тем выше оценка.

Преимущества:

- подходит для больших многомерных наборов данных;
- не требует вычисления всех попарных расстояний;
- не предполагает конкретного распределения;
- является одним из популярных базовых методов обнаружения аномалий.

Ограничения:

- результат зависит от параметра ожидаемой доли аномалий;
- сложные контекстуальные зависимости могут быть не обнаружены;
- категориальные признаки требуют предварительного кодирования;
- оценку модели не всегда легко объяснить аналитику.

8. Автоэнкодеры.

Автоэнкодер — нейронная сеть, которая обучается восстанавливать входные данные.

Модель состоит из двух основных частей:

- кодировщик преобразует входные данные в компактное представление;
- декодировщик восстанавливает исходные данные.

Если автоэнкодер обучен преимущественно на нормальных событиях, он хорошо восстанавливает нормальные объекты и хуже — неизвестные или аномальные.

Если ошибка превышает порог, объект считается аномальным.

Преимущества:

- обнаружение сложных нелинейных зависимостей;
- возможность работы с большим количеством признаков;
- применимость к сетевому трафику, журналам, изображениям и последовательностям.

Ограничения:

- требуется достаточно большой объём данных;
- необходимо настраивать архитектуру и порог;
- результат сложно интерпретировать;
- мощная модель иногда хорошо восстанавливает даже аномальные объекты;
- обучение может потребовать значительных вычислительных ресурсов.

9. Методы анализа временных рядов.

Если события имеют временную структуру, важно учитывать не только значения признаков, но и их изменение во времени.

Применяются:

- скользящее среднее;
- экспоненциальное сглаживание;
- ARIMA;
- LSTM;
- трансформеры.

Эти методы позволяют обнаруживать:

- резкие всплески;
- необычные падения активности;
- изменения тренда;
- нарушение периодичности;
- постепенный дрейф поведения;
- нетипичные последовательности событий.

Например, 20 запросов в минуту могут быть нормальными сами по себе. Но резкий переход от 2 до 20 запросов для конкретного пользователя может быть аномальным изменением.

Представленная классификация не является строгой, так как в большинстве случаев один о тот же метод можно отнести к разным группам.

4. Описание исходных данных

4.1.Организация эксперимента

Каждая строка таблицы — это часовой профиль активности на хосте (или шлюзе). Признаки описывают, как вёл себя именно этот узел: сколько было неудачных входов, какой был средний трафик, сколько уникальных адресатов и т. д. Такая агрегация удобна для пакетного анализа и приоритизации окон под ручной разбор. В выборку не смешиваются разные пользователи и устройства.

Признак	Содержание и единица
`timestamp`	Начало часового интервала
`failed_logins`	Число неудачных входов за час; целое неотрицательное число
`mean_session_min`	Средняя продолжительность сеансов, представленных в часовом агрегате, в минутах
`bytes_out_mb`	Общий исходящий объём за час, МБ
`mean_connections_per_min`	Среднее число соединений в минуту за рассматриваемый час
`mean_unique_destinations`	Среднее по минутам число уникальных адресатов за час
`mean_requests_per_min`	Средняя интенсивность запросов за час, запросов в минуту

Средние количества и интенсивности могут быть дробными: например, 120,5 соединения в минуту означает среднее за час, а не половину отдельного соединения. `failed_logins`, наоборот, является суммарным счётчиком и остаётся целым.

Создаются 90 суток наблюдений. Первые 30 суток — обучение, следующие 30 — настройка порога, последние 30 — проверка. Размер выбран для наглядности, а не как универсально достаточная длина истории.

In [1]:
```
import sys
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy
from scipy import stats
import sklearn
from sklearn.covariance import MinCovDet
from IPython.display import display

# Фиксируем читаемый вывод чисел, не меняя точность самих вычислений.
pd.set_option("display.max_columns", 12)
pd.set_option("display.width", 150)
pd.set_option("display.float_format", lambda value: f"{value:.4f}")
```

In [2]:
```
# @title
# Фиксированный seed делает набор значений воспроизводимым.
rng = np.random.default_rng(20261017)
HOURS_PER_PART = 30 * 24
n_rows = 3 * HOURS_PER_PART

data = pd.DataFrame({
    "timestamp": pd.date_range("2026-01-01", periods=n_rows, freq="h"),
    "failed_logins": rng.poisson(lam=1.2, size=n_rows),
    "mean_session_min": np.clip(rng.normal(35, 6, n_rows), 5, None),
    "bytes_out_mb": rng.lognormal(mean=2.8, sigma=0.65, size=n_rows),
    "mean_connections_per_min": rng.normal(120, 18, n_rows),
    "mean_requests_per_min": rng.normal(100, 9, n_rows),
})
```

```python
# Связанные сетевые признаки: адресатов обычно больше при большем
# количестве соединений. Шум означает естественные колебания этой связи.
data["mean_unique_destinations"] = (
    5 + 0.065 * data["mean_connections_per_min"]
    + rng.normal(0, 1.5, n_rows)
)

# Истину храним отдельно: эти столбцы не являются входами моделей.
TARGET_COLUMNS = {
    "long_session": "attack_long_session",
    "exfiltration": "attack_exfiltration",
    "brute_force": "attack_brute_force",
    "joint": "attack_joint",
    "shift": "attack_shift",
}
truth = pd.DataFrame(False, index=data.index,
                     columns=list(TARGET_COLUMNS.values()))
truth["legitimate_note"] = "обычная штатная работа"

# В каждом временном интервале есть сильные и умеренные целевые события.
# Их позиции известны автору генератора, но не передаются функциям fit.
for part in range(3):
    start = part * HOURS_PER_PART

    # Законные редкие операции сохраняются как нормальные события.
    for offset, value in [(90, 130.0), (500, 180.0)]:
        data.loc[start + offset, "bytes_out_mb"] = value
        truth.loc[start + offset, "legitimate_note"] = "разрешённый экспорт отчё
    for offset, value in [(82, 60.0), (480, 58.0)]:
        data.loc[start + offset, "mean_session_min"] = value
        truth.loc[start + offset, "legitimate_note"] = "плановое длительное обсл
    for offset, value in [(95, 5), (510, 6)]:
        data.loc[start + offset, "failed_logins"] = value
        truth.loc[start + offset, "legitimate_note"] = "законные ошибки входа"

    # Длительные сеансы: обучение намеренно загрязнено сильными значениями.
    session_values = [[220, 250, 70, 300], [48, 55, 72, 95], [47, 57, 75, 105]][
    for offset, value in zip([50, 210, 380, 610], session_values):
        data.loc[start + offset, "mean_session_min"] = value
        truth.loc[start + offset, "attack_long_session"] = True

    # Утечки представлены разными объёмами: умеренные сложнее отличить
    # от правого хвоста штатного логнормального трафика.
    byte_values = [[45, 75, 420, 680], [38, 60, 220, 500], [40, 70, 240, 600]][p
    for offset, value in zip([110, 260, 440, 650], byte_values):
        data.loc[start + offset, "bytes_out_mb"] = value
        truth.loc[start + offset, "attack_exfiltration"] = True

    login_values = [[9, 14, 6, 22], [7, 12, 5, 18], [8, 11, 5, 20]][part]
    for offset, value in zip([35, 180, 360, 600], login_values):
        data.loc[start + offset, "failed_logins"] = value
        truth.loc[start + offset, "attack_brute_force"] = True

    # Обучающие сетевые аномалии сильно искажают обычную ковариацию.
    # На калибровке и проверке сочетания умереннее и ближе к норме.
    joint_values = (
        [(140, 45), (130, 55), (145, 50), (125, 60)] if part == 0
        else [(140, 9), (145, 10), (100, 17), (130, 10.7)]
    )
```

```
    for offset, values in zip([65, 240, 420, 630], joint_values):
        data.loc[start + offset, ["mean_connections_per_min", "mean_unique_desti
        truth.loc[start + offset, "attack_joint"] = True

    # Два устойчивых сдвига интенсивности. Это целевые окна одного процесса,
    # а не 36 независимых инцидентов. В основном сравнении они не участвуют.
    for first, last, increase in [(280, 304, 14), (550, 562, 20)]:
        indices = np.arange(start + first, start + last)
        data.loc[indices, "mean_requests_per_min"] += increase
        truth.loc[indices, "attack_shift"] = True

# Отдельно помечаем целевые строки в пояснении, чтобы не называть
# внедрённое событие штатным. Булевы метки остаются источником истины.
has_injected_scenario = truth[list(TARGET_COLUMNS.values())].any(axis=1)
truth.loc[has_injected_scenario, "legitimate_note"] = "внедрённый целевой атакую

# Округление соответствует точности учебной записи, а не оценке модели.
float_features = [column for column in data if column not in ["timestamp", "fail
data[float_features] = data[float_features].round(3)

print("Наблюдений:", len(data))
data.head(2)
```

```
Наблюдений: 2160
```

Out[2]:

	timestamp	failed_logins	mean_session_min	bytes_out_mb	mean_connections_per_mi
0	2026-01-01 00:00:00	4	35.3390	11.8850	98.898
1	2026-01-01 01:00:00	1	31.4570	31.8130	101.891

Разделим исходные данные на три части: `train`, `calibration` и `test`.

Каждая из этих выборок будет иметь одинаковый размер:

- 30 — количество суток;
- 24 — количество часов в сутках;
- результат — 720 часов.

In [3]:
```
# Количество часов в каждой выборке.
HOURS_PER_PART
```

Out[3]:
```
720
```

In [4]:
```
# Истину храним отдельно: эти столбцы не являются входами моделей.
TARGET_COLUMNS
```

Out[4]:
```
{'long_session': 'attack_long_session',
 'exfiltration': 'attack_exfiltration',
 'brute_force': 'attack_brute_force',
 'joint': 'attack_joint',
 'shift': 'attack_shift'}
```

`truth` — отдельная таблица с известной истиной: какие сценарии действительно внедрены в каждое наблюдение. В лекции используются две таблицы:

Таблица	Что хранит	Для чего используется
`data`	Измеренные признаки: объём трафика, ошибки входа, длительности и т. д.	Детектор анализирует эти значения
`truth`	Метки внедрённых сценариев и пояснения	Проверяем, что детектор обнаружил правильно, ложно или пропустил

Создадим две обучающие выборки — `train` для обучения на неразмеченной истории (норма + атаки) и `normal_train` для обучения только на норме (только норма без атак). Калибровочная и тестовая выборки общие для обоих режимов.

In [5]:
```
# Разделяем время без перемешивания: будущее не участвует в оценке прошлого.
train = data.iloc[:HOURS_PER_PART].copy()
calibration = data.iloc[HOURS_PER_PART:2 * HOURS_PER_PART].copy()
test = data.iloc[2 * HOURS_PER_PART:].copy()

# Только при подготовке режима «норма» используем скрытую истину.
# Исключаем все внедрённые атаки, но сохраняем законные редкие операции.
attack_columns = list(TARGET_COLUMNS.values())
normal_training_mask = ~truth.loc[train.index, attack_columns].any(axis = 1)
normal_train = train.loc[normal_training_mask].copy()

phase_rows = []
for name, frame in [("Смешанная обучающая выборка", train),
                    ("Нормальная обучающая выборка", normal_train),
                    ("Калибровочная выборка", calibration),
                    ("Тестовая выборка", test)]:
    row = {"Интервал": name, "Начало": frame["timestamp"].min(),
           "Конец": frame["timestamp"].max(), "Наблюдений": len(frame)}
    row.update({key: int(truth.loc[frame.index, column].sum())
                for key, column in TARGET_COLUMNS.items()})
    phase_rows.append(row)
pd.DataFrame(phase_rows)
```

Out[5]:

	Интервал	Начало	Конец	Наблюдений	long_session	exfiltration	brute_for
0	Смешанная обучающая выборка	2026-01-01	2026-01-30 23:00:00	720	4	4	
1	Нормальная обучающая выборка	2026-01-01	2026-01-30 23:00:00	668	0	0	
2	Калибровочная выборка	2026-01-31	2026-03-01 23:00:00	720	4	4	
3	Тестовая выборка	2026-03-02	2026-03-31 23:00:00	720	4	4	

Каждый временной интервал содержит 720 часов. Столбцы `long_session`, `exfiltration`, `brute_force`, `joint`, `shift` показывают количество внедрённых атак и используются только для проверки качества детекции. В каждом интервале по четыре атаки каждого типа и 36 окон устойчивого сдвига.

Нормальная обучающая история короче исходной: из неё удалены строки с внедрёнными атаками.

История «на норме» содержит законные экспорты и обслуживание. Отсутствие атак не означает отсутствия редких штатных значений.

Колибровочная выборка будет использоваться только для подбора порога аномальности объектов.

4.2.Целевой сценарий обнаружения аномалий

Задать целевой сценарий означает заранее определить, какие события должен находить конкретный детектор. Например: «Этот детектор ищет возможные утечки по необычно большому исходящему объёму».

Рассмотрим условный пример. Одна строка — один час работы узла:

Что произошло	**Исходящий трафик** `bytes_out_mb`	**Ошибки входа** `failed_logins`
Штатная работа	20 МБ	1
Подбор пароля	20 МБ	30
Утечка данных	600 МБ	1

Используем два простых детектора:

- детектор объёма: тревога, если исходящий трафик больше 100 МБ;
- детектор ошибок входа: тревога, если ошибок больше 10.

Получаем:

Что произошло	**Детектор объёма**	**Детектор ошибок входа**
Штатная работа	Нет тревоги	Нет тревоги
Подбор пароля	Нет тревоги	**Тревога**
Утечка данных	**Тревога**	Нет тревоги

Поэтому при проверке задаём конкретные вопросы:

- для детектора объёма: «Сколько внедрённых утечек он обнаружил и сколько законных передач отметил ошибочно?»;
- для детектора ошибок: «Сколько случаев подбора пароля он обнаружил и сколько штатных часов отметил ошибочно?».

Но если задача сформулирована шире — «обнаруживать все атаки», — незамеченный подбор пароля будет пропуском всей системы. Тогда одного детектора объёма недостаточно: необходимо использовать дополнительные признаки или несколько детекторов.

В ходе детекции аномалий возможны следующие сценарии:

Показатель	Смысл
Всего целевых событий	Число наблюдений с выбранным целевым сценарием
Срабатываний	Сколько наблюдений отметил детектор
Правильно обнаружено	Тревога есть и выбранный целевой сценарий есть
Ложно обнаружено	Тревога есть, а выбранного целевого сценария нет
Пропущено	Целевой сценарий есть, а тревоги нет

Таким образом, «ложно» в таблицах означает ошибку относительно указанного сценария. На такой строке может происходить другая атака, которая не проявилась в анализируемых признаках. Совпадение тревоги с чужим сценарием не засчитывается как подтверждённое обнаружение его механизма.

5. Статистические методы выявления аномалий

Статистические методы представляют собой фундаментальный класс подходов к обнаружению аномалий, основанный на предположении, что нормальное поведение подчиняется определенным статистическим закономерностям. Эти методы особенно эффективны, когда данные имеют относительно стабильное распределение и аномалии проявляются как статистические выбросы.

Наблюдение считается аномальным, если оно:

- слишком сильно отличается от среднего или медианного значения;
- выходит за границы типичного диапазона;
- имеет низкую вероятность появления;
- нарушает характерную взаимосвязь между признаками;
- указывает на изменение статистических свойств процесса во времени.

5.1. Z-score и правило трёх стандартных отклонений (3σ)

По обучающей выборке рассчитываются:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}.$$

Здесь:

- n — число обучающих наблюдений;
- i — номер наблюдения;
- x_i — значение признака у i-го наблюдения;
- $\bar{x}$ — выборочное среднее;
- s — выборочное стандартное отклонение.

Среднее описывает центр, стандартное отклонение — масштаб колебаний. Делитель $n - 1$ даёт несмещённую оценку дисперсии s^2 при обычных предположениях случайной выборки.

Для нового значения x оценка определяется как

$$z_i = \frac{x_i - \bar{x}}{s}.$$

Положительная Z-оценка означает значение выше обучающего среднего; отрицательная — ниже. Например, $z = 3$ означает превышение среднего на три стандартных отклонения, а не вероятность атаки.

Общее правило принятия решения об аномальности объекта:

$$z_i > T \quad \Longleftrightarrow \quad x_i > \bar{x} + T \cdot s,$$

где $T > 0$ — выбранный порог аномальности Z-оценки. При двусторонней задаче используют $|z_i| > T$ и проверяют обе стороны.

Если данные подчиняются нормальному закону распределения, то:

- около 68 % значений находятся в пределах $\bar{x} \pm s$;
- около 95 % — в пределах $\bar{x} \pm 2s$;
- около 99,7 % — в пределах $\bar{x} \pm 3s$.

Отсюда следует правило:

$$|z_i| > 3 \Rightarrow x_i \text{ является потенциальной аномалией.}$$

Доля около 0,3 % за границами трёх сигм относится к **двум хвостам**. В задачах кибербезопасности часто интересует только верхняя граница. Например, большое число ошибок входа опасно, а нулевое число ошибок является нормальным. При контроле только верхнего хвоста нормальной модели вероятность превышения меньше, около 0,135 %.

Эти теоретические вероятности не заменяют подсчёт реальных ошибок и не обещают такого же результата при оценённых параметрах, зависимости наблюдений и изменении режима.

Z-score можно вычислить для любого распределения. Но для сильно асимметричного распределения стандартный порог 3 является эвристикой. Сама возможность выполнить деление не означает пригодности правила для задачи.

Рассмотрим пример использования метода для поиска аномалий в значениях признака `mean_session_min`. В нашем сценарии требуется выявлять необычно длительные сеансы. Сокращение продолжительности в этой задаче не является целевым признаком атаки, поэтому будем использовать одностороннее правило.

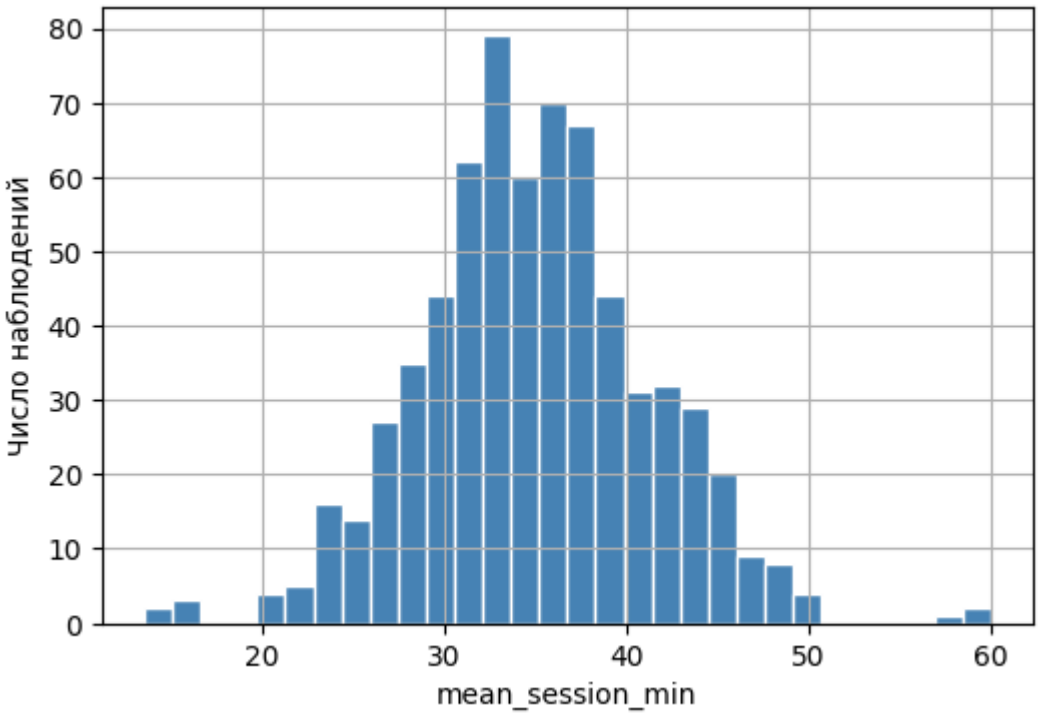
Технологии реализации метода будем рассматривать для двух режимов:

- **обучение на норме:** среднее и стандартное отклонение оцениваются по представительной нормальной выборке и фиксируются;
- **обучение на неразмеченной смешанной выборке:** используются те же формулы, но наличие атак может сместить центр и увеличить разброс. Относительно большие значения признака расширяют границу аномальности и скрывают менее выраженные отклонения — возникает эффект маскировки.

Z-score в режиме обучения только на норме

Визуальный анализ распределения признака.

In [6]:
```
plt.figure(figsize = (6, 4))
plt.hist(normal_train["mean_session_min"], bins = 30,
         edgecolor = "white", color = "steelblue")
plt.xlabel ('mean_session_min')
plt.ylabel ("Число наблюдений")
plt.grid();
```



Для признака `mean_session_min` метод Z-score применять допустимо: распределение на гистограмме близко к нормальному (симметричный «колокол», умеренные хвосты), что соответствует предпосылке Z-score о нормальности данных.

Шаг 1. Оцениваем параметры z-оценки.

Используем выборку `normal_train`.

In [7]:
```
# Непосредственно реализуем обучающие среднее и std из формулы Z-score.
z_values_normal = normal_train["mean_session_min"].to_numpy(dtype = float)
z_mean_normal = z_values_normal.mean()
z_std_normal = z_values_normal.std(ddof = 1)
pd.Series({"Среднее, мин.": z_mean_normal,
           "Стандартное отклонение, мин.": z_std_normal}).to_frame("Параметры")

z_normal_threshold = 3
z_normal_raw_boundary = z_mean_normal + z_normal_threshold * z_std_normal
print("Граница оценки:", z_normal_threshold)
print(f'Граница в исходной шкале: {round(z_normal_raw_boundary, 2)} мин.')
```

```
Граница оценки: 3
Граница в исходной шкале: 53.77 мин.
```

Шаг 2. Применяем заранее заданное правило к новым данным.

Для объектов тестовой выборки `test`, рассчитываем z-оценки на основе вычисленных на нормальной обучающей выборке параметров и сравниваем с зафиксированной границей.

In [8]:
```
# Устанавливаем пороговое значение Z-score:
# наблюдения со значением выше этого порога считаются аномальными.
z_normal_threshold = 3

# Вычисляем Z-оценки для признака mean_session_min в тестовой выборке.
z_normal_scores_test = (test["mean_session_min"].to_numpy() - z_mean_normal) / z

# Формируем булев массив тревог:
# True, если Z-оценка превышает порог (потенциальная аномалия).
z_normal_alerts_test = z_normal_scores_test > z_normal_threshold
print(f'Количество тревог: {z_normal_alerts_test.sum()}')
```

```
Количество тревог: 5
```

Шаг 3. Оцениваем качество работы метода.

После получения тревог присоединяем известную истину для подсчёта ошибок.

Для оценивания качества детекции нужно определить целевой сценарий работы детектора и метрики.

Целевой сценарий — это заранее определённая ситуация, которую требуется обнаруживать. Она не обязательно является атакой. Например, целью может быть обнаружение технического сбоя, ошибочной настройки или необычного увеличения нагрузки.

В нашем примере целевой сценарий — `long_session`: требуется обнаруживать внедрённое атакующее поведение, которое увеличивает длительность сеансов.

Качество детекции будем оценивать с помощью следующих метрик:

- количество верно обнаруженных атак;
- количество ложных срабатываний;

- количество пропущенных атак.

In [9]:
```
# Извлекаем истинные метки для тестовой выборки по признаку attack_long_session.
z_normal_target_test = truth.loc[test.index, "attack_long_session"].to_numpy()

# Считаем количество верно обнаруженных атак:
# тревога сработала и атака действительно была.
z_normal_found = int((z_normal_alerts_test & z_normal_target_test).sum())

# Считаем количество ложных срабатываний: тревога сработала, но атаки не было.
z_normal_false = int((z_normal_alerts_test & ~z_normal_target_test).sum())

# Считаем количество пропущенных атак: тревога не сработала, но атака была.
z_normal_missed = int((~z_normal_alerts_test & z_normal_target_test).sum())

z_normal_report = pd.Series({"Всего атак": int(z_normal_target_test.sum()),
                             "Срабатываний": int(z_normal_alerts_test.sum()),
                             "Правильно обнаружено": z_normal_found,
                             "Ложно обнаружено": z_normal_false,
                             "Пропущено": z_normal_missed,},
                             name = "Z-score — на нормальной выборке")
z_normal_report.to_frame()
```

Out[9]:

	Z-score — на нормальной выборке
Всего атак	4
Срабатываний	5
Правильно обнаружено	3
Ложно обнаружено	2
Пропущено	1

В тестовой выборке имеются четыре наблюдения с внедрённым сценарием атаки (увеличение длительности сеансов). Метод обнаружил три из них и пропустил одну. Ещё для двух наблюдений он сформировал тревоги, хотя этого сценария там не было. Всего сформировано пять тревог.

Отобразим наблюдения, на которых детектор правильно обнаружил атаки, выдал ложную тревогу и пропустил атаку.

In [10]:
```
z_normal_events = test[["timestamp", 'mean_session_min']].copy()
z_normal_events["Оценка"] = z_normal_scores_test
z_normal_events["Атака"] = z_normal_target_test
z_normal_events["Тревога"] = z_normal_alerts_test
z_normal_events["Сведения генератора"] = truth.loc[test.index, "legitimate_note"
z_normal_target_examples = z_normal_events.loc[z_normal_target_test]
z_normal_false_examples = z_normal_events.loc[z_normal_alerts_test & ~z_normal_t
display(pd.concat([z_normal_target_examples, z_normal_false_examples]).sort_valu
```

	timestamp	mean_session_min	Оценка	Атака	Тревога	Сведения генератора
1490	2026-03-04 02:00:00	47.0000	1.9208	True	False	внедрённый целевой атакующий сценарий
1522	2026-03-05 10:00:00	60.0000	3.9935	False	True	плановое длительное обслуживание
1650	2026-03-10 18:00:00	57.0000	3.5152	True	True	внедрённый целевой атакующий сценарий
1820	2026-03-17 20:00:00	75.0000	6.3851	True	True	внедрённый целевой атакующий сценарий
1920	2026-03-22 00:00:00	58.0000	3.6746	False	True	плановое длительное обслуживание
2050	2026-03-27 10:00:00	105.0000	11.1682	True	True	внедрённый целевой атакующий сценарий

Z-score в режиме обучения на неразмеченной выборке

Шаг 1. Оцениваем параметры z-оценки.

Используем смешанную обучающую выборку `train`, которая содержит как нормальные наблюдения, так и наблюдения с атакующими сценариями.

In [11]:
```
z_values_unlabelled = train["mean_session_min"].to_numpy(dtype = float)
z_mean_unlabelled = z_values_unlabelled.mean()
z_std_unlabelled = z_values_unlabelled.std(ddof = 1)

pd.Series({"Среднее, мин.": z_mean_unlabelled,
           "Стандартное отклонение, мин.": z_std_unlabelled}).to_frame("Параметр

z_unlabelled_threshold = 3
z_unlabelled_raw_boundary = z_mean_unlabelled + z_unlabelled_threshold * z_std_u
print("Граница оценки:", z_unlabelled_threshold)
print(f'Граница в исходной шкале: {round(z_unlabelled_raw_boundary, 2)} мин.')
```

```
Граница оценки: 3
Граница в исходной шкале: 83.35 мин.
```

Шаг 2. Применяем заранее заданное правило к новым данным.

Для объектов тестовой выборки `test`, рассчитываем z-оценки на основе вычисленных на неразмеченной обучающей выборке параметров и сравниваем с

зафиксированной границей.

In [12]:
```
z_unlabelled_scores_test = (test["mean_session_min"].to_numpy() - z_mean_unlabel
z_unlabelled_alerts_test = z_unlabelled_scores_test > z_unlabelled_threshold
print(f'Количество тревог: {z_unlabelled_alerts_test.sum()}')
```

```
Количество тревог: 1
```

Шаг 3. Оцениваем качество работы метода.

После получения тревог присоединяем известную истину для подсчёта ошибок.

Целевой сценарий — `long_session` : требуется обнаруживать внедрённое атакующее поведение, которое увеличивает длительность сеансов.

Качество детекции будем оценивать с помощью следующих метрик:

- количество верно обнаруженных атак;
- количество ложных срабатываний;
- количество пропущенных атак.

In [13]:
```
z_unlabelled_target_test = truth.loc[test.index, "attack_long_session"].to_numpy

# Каждое выражение сопоставляет тревогу и истину одного и того же наблюдения.
z_unlabelled_found = int((z_unlabelled_alerts_test & z_unlabelled_target_test).s
z_unlabelled_false = int((z_unlabelled_alerts_test & ~z_unlabelled_target_test).
z_unlabelled_missed = int((~z_unlabelled_alerts_test & z_unlabelled_target_test)

z_unlabelled_report = pd.Series({"Всего атак": int(z_unlabelled_target_test.sum(
                                 "Срабатываний": int(z_unlabelled_alerts_test.su
                                 "Правильно обнаружено": z_unlabelled_found,
                                 "Ложно обнаружено": z_unlabelled_false,
                                 "Пропущено": z_unlabelled_missed},
                                 name = "Z-score — на неразмеченной выборке")
z_unlabelled_report.to_frame()
```

Out[13]:

	Z-score — на неразмеченной выборке
Всего атак	4
Срабатываний	1
Правильно обнаружено	1
Ложно обнаружено	0
Пропущено	3

Отобразим наблюдения, на которых детектор правильно обнаружил атаки, выдал ложную тревогу и пропустил атаку.

In [14]:
```
z_unlabelled_events = test[["timestamp", 'mean_session_min']].copy()
z_unlabelled_events["Оценка"] = z_unlabelled_scores_test
z_unlabelled_events["Целевой сценарий"] = z_unlabelled_target_test
z_unlabelled_events["Тревога"] = z_unlabelled_alerts_test
z_unlabelled_events["Сведения генератора"] = truth.loc[test.index, "legitimate_n
z_unlabelled_target_examples = z_unlabelled_events.loc[z_unlabelled_target_test]
```

```
z_unlabelled_false_examples = z_unlabelled_events.loc[z_unlabelled_alerts_test &
pd.concat([z_unlabelled_target_examples, z_unlabelled_false_examples]).sort_valu
```

Out[14]:

	timestamp	mean_session_min	Оценка	Целевой сценарий	Тревога	Сведения генератора
1490	2026-03-04 02:00:00	47.0000	0.6970	True	False	внедрённый целевой атакующий сценарий
1650	2026-03-10 18:00:00	57.0000	1.3304	True	False	внедрённый целевой атакующий сценарий
1820	2026-03-17 20:00:00	75.0000	2.4707	True	False	внедрённый целевой атакующий сценарий
2050	2026-03-27 10:00:00	105.0000	4.3712	True	True	внедрённый целевой атакующий сценарий

Z-score: сопоставление двух режимов

В обоих режимах использовано одно заранее заданное правило. Различия возникают из-за состава истории и оценённых по ней параметров. Сначала сопоставим параметры, затем фактические решения.

In [15]:
```
z_comparison = pd.concat([z_normal_report, z_unlabelled_report], axis = 1).T
z_comparison
```

Out[15]:

	Всего атак	Срабатываний	Правильно обнаружено	Ложно обнаружено	Пропущено
Z-score — на нормальной выборке	4	5	3	2	1
Z-score — на неразмеченной выборке	4	1	1	0	3

In [16]:
```
pd.DataFrame({"Выборка": ["Нормальная", "Неразмеченная"],
              "Среднее, мин.": [z_mean_normal, z_mean_unlabelled],
              "Std, мин.": [z_std_normal, z_std_unlabelled],
              "Выбранная граница, мин.": [z_mean_normal + z_normal_threshold * z
                                          z_mean_unlabelled + z_unlabelled_thres
```

Out[16]:

	Выборка	Среднее, мин.	Std, мин.	Выбранная граница, мин.
0	Нормальная	34.9526	6.2720	53.7687
1	Неразмеченная	35.9983	15.7855	83.3548

- **Z-score на нормальной выборке**:
  - высокая чувствительность: 3 из 4 атак найдено, но 2 ложных срабатывания;
  - узкое распределение нормы позволяет точнее локализовать аномалии, но часть «пограничных» атак остаётся вне порога.
- **Z-score на неразмеченной выборке**:
  - из-за включения аномалий в расчёт σ разброс резко вырос — порог стал слишком высоким;
  - результат: только 1 из 4 атак найдено, 0 ложных срабатываний;
  - большинство аномалий не превышают порог из-за «размывания» статистики.

Вывод: для метода Z-score критично обучение на чистой норме — неразмеченные данные искажают σ и снижают количество выявленных атак.

Ограничения метода.

1. Предполагает приблизительную симметричность данных.
2. Среднее и стандартное отклонение чувствительны к выбросам. Сильная аномалия увеличивает стандартное отклонение и может скрыть менее выраженные аномалии. Это называется эффектом маскировки.
3. Универсальный порог 3 не учитывает допустимое число предупреждений в конкретной организации.
4. Метод анализирует признаки по отдельности и не обнаруживает необычные сочетания признаков.
5. Нормальное значение может зависеть от контекста. Десять ошибок входа для одного пользователя и десять ошибок для всей организации имеют разный смысл.
6. Один общий порог может быть неправильным для разных групп устройств. Например, нормальный трафик сервера и рабочей станции существенно различается.

5.2. Модифицированный Z-score

Обычный Z-score использует среднее и стандартное отклонение, чувствительные к выбросам.

Робастной альтернативой этих числовых характеристик являются:

- для среднего - медиана;

- для стандартного отклонения - медианное абсолютное отклонение — MAD.

Медиана — центральное значение упорядоченной выборки; для чётного числа элементов обычно усредняются два центральных. Она зависит от порядка значений, поэтому одно очень большое значение обычно влияет на неё значительно меньше, чем на среднее.

MAD (median absolute deviation, медианное абсолютное отклонение) характеризует типичное абсолютное удаление от медианы:

$$\mathrm{MAD} = \mathrm{median}\left(|x_i - \mathrm{median}(x)|\right).$$

Модифицированный Z-score определяется так:

$$M_i = \frac{c \cdot (x_i - \mathrm{median}(x))}{\mathrm{MAD}}, \qquad c \approx 0{,}67449.$$

Константа c — 0,75-квантиль стандартного нормального распределения. Для нормального закона MAD примерно равен $0{,}67449\,\sigma$, где σ — стандартное отклонение. Поэтому множитель согласует масштабы M и обычной Z-оценки при нормальном распределении.

Для анализа верхнего хвоста распределения используется общее правило $M_i > T$. Соответствующая граница в исходных единицах:

$$x > \mathrm{median}(x) + \frac{T \cdot \mathrm{MAD}}{c}.$$

Для двустороннего поиска используется правило $|M_i| > T$.

По умолчанию применяется порог $k = 3,5$:

$$|M_i| > 3,5.$$

Метод особенно полезен, когда данные загрязнены экстремальными выбросами, — в отличие от стандартного Z-score, MAD устойчив к влиянию сильных выбросов за счёт использования медианы вместо среднего и абсолютных отклонений.

Модифицированный Z-score в режиме обучения только на норме

Шаг 1. Оцениваем параметры.

Используем `normal_train`. Метки калибровочных и проверочных событий в расчёте параметров не участвуют.

In [17]:
```
# Константа c.
MAD_CONSTANT = stats.norm.ppf(0.75)

mad_values_normal = normal_train["mean_session_min"].to_numpy(dtype = float)
mad_median_normal = np.median(mad_values_normal)
```

```
# Сначала абсолютное отклонение каждой длительности от медианы,
# затем медиана этих отклонений.
mad_absolute_deviations_normal = np.abs(mad_values_normal - mad_median_normal)
mad_scale_normal = np.median(mad_absolute_deviations_normal)

display(pd.Series({"Медиана, мин.": mad_median_normal, "MAD, мин.": mad_scale_no

mad_normal_threshold = 3.5
mad_normal_raw_boundary = mad_median_normal + mad_normal_threshold * mad_scale_n
print("Граница оценки:", mad_normal_threshold)
print("Граница в исходной шкале:", round(mad_normal_raw_boundary, 2))
```

	Параметры
Медиана, мин.	34.7615
MAD, мин.	3.9245

```
Граница оценки: 3.5
Граница в исходной шкале: 55.13
```

Шаг 2. Применяем заранее заданное правило к новым данным.

Для объектов тестовой выборки `test` рассчитываем модифицированные z-оценки на основе вычисленных на нормальной обучающей выборке параметров и сравниваем с зафиксированной границей.

In [18]:
```
mad_normal_scores_test = MAD_CONSTANT * (test["mean_session_min"].to_numpy() - m
mad_normal_alerts_test = mad_normal_scores_test > mad_normal_threshold
print(f'Количество тревог: {mad_normal_alerts_test.sum()}')
```

```
Количество тревог: 5
```

Шаг 3. Оцениваем качество работы метода.

После получения тревог присоединяем известную истину для подсчёта ошибок.

Целевой сценарий — `long_session`: требуется обнаруживать внедрённое атакующее поведение, которое увеличивает длительность сеансов.

Качество детекции будем оценивать с помощью следующих метрик:

- количество верно обнаруженных атак;
- количество ложных срабатываний;
- количество пропущенных атак.

In [19]:
```
mad_normal_target_test = truth.loc[test.index, "attack_long_session"].to_numpy()

# Каждое выражение сопоставляет тревогу и истину одного и того же часа.
mad_normal_found = int((mad_normal_alerts_test & mad_normal_target_test).sum())
mad_normal_false = int((mad_normal_alerts_test & ~mad_normal_target_test).sum())
mad_normal_missed = int((~mad_normal_alerts_test & mad_normal_target_test).sum()

mad_normal_report = pd.Series({"Всего атак": int(mad_normal_target_test.sum()),
                               "Срабатываний": int(mad_normal_alerts_test.sum())
                               "Правильно обнаружено": mad_normal_found,
                               "Ложно обнаружено": mad_normal_false,
```

```
                                "Пропущено": mad_normal_missed},
                                name = "Модифицированный Z-score — на нормальной
mad_normal_report.to_frame()
```

Out[19]:

	Модифицированный Z-score — на нормальной выборке
Всего атак	4
Срабатываний	5
Правильно обнаружено	3
Ложно обнаружено	2
Пропущено	1

Отобразим наблюдения, на которых детектор правильно обнаружил атаки, выдал ложную тревогу и пропустил атаку.

In [20]:
```
mad_normal_events = test[["timestamp", 'mean_session_min']].copy()
mad_normal_events["Оценка"] = mad_normal_scores_test
mad_normal_events["Целевой сценарий"] = mad_normal_target_test
mad_normal_events["Тревога"] = mad_normal_alerts_test
mad_normal_events["Сведения генератора"] = truth.loc[test.index, "legitimate_not
mad_normal_target_examples = mad_normal_events.loc[mad_normal_target_test]
mad_normal_false_examples = mad_normal_events.loc[mad_normal_alerts_test & ~mad_
display(pd.concat([mad_normal_target_examples, mad_normal_false_examples]).sort_
```

	timestamp	mean_session_min	Оценка	Целевой сценарий	Тревога	Сведения генератора
1490	2026-03-04 02:00:00	47.0000	2.1034	True	False	внедрённый целевой атакующий сценарий
1522	2026-03-05 10:00:00	60.0000	4.3377	False	True	плановое длительное обслуживание
1650	2026-03-10 18:00:00	57.0000	3.8221	True	True	внедрённый целевой атакующий сценарий
1820	2026-03-17 20:00:00	75.0000	6.9156	True	True	внедрённый целевой атакующий сценарий
1920	2026-03-22 00:00:00	58.0000	3.9939	False	True	плановое длительное обслуживание
2050	2026-03-27 10:00:00	105.0000	12.0716	True	True	внедрённый целевой атакующий сценарий

Таким образом, на нормальной истории обычный и модифицированный методы Z-score дают одинаковое качество детекции. Это обусловлено отсутствием в нормальной выборке экстремальных значений, что приводит к примерно одинаковым пороговым значениям: ≈ 53.77 минут и ≈ 55.13 минут.

Модифицированный Z-score в режиме обучения на неразмеченной выборке

Шаг 1. Оцениваем параметры.

Используем смешанную выборку `train`.

In [21]:
```
mad_values_unlabelled = train["mean_session_min"].to_numpy(dtype = float)
mad_median_unlabelled = np.median(mad_values_unlabelled)

# Сначала абсолютное отклонение каждой длительности от медианы,
# затем медиана этих отклонений.
mad_absolute_deviations_unlabelled = np.abs(mad_values_unlabelled - mad_median_u
mad_scale_unlabelled = np.median(mad_absolute_deviations_unlabelled)

display(pd.Series({"Медиана, мин.": mad_median_unlabelled, "MAD, мин.": mad_scal

mad_unlabelled_threshold = 3.5
mad_unlabelled_raw_boundary = mad_median_unlabelled + mad_unlabelled_threshold *
```

```
print("Граница оценки:", mad_unlabelled_threshold)
print("Граница в исходной шкале:", round(mad_unlabelled_raw_boundary, 2))
```

	Параметры
Медиана, мин.	34.8905
MAD, мин.	3.9930

```
Граница оценки: 3.5
Граница в исходной шкале: 55.61
```

Шаг 2. Применяем заранее заданное правило к новым данным.

Для объектов тестовой выборки `test`, рассчитываем модифицированные z-оценки на основе вычисленных на неразмеченной обучающей выборке параметров и сравниваем с зафиксированной границей.

In [22]:
```
mad_unlabelled_scores_test = MAD_CONSTANT * (test["mean_session_min"].to_numpy()
mad_unlabelled_alerts_test = mad_unlabelled_scores_test > mad_unlabelled_thresho
print(f'Количество тревог: {mad_unlabelled_alerts_test.sum()}')
```

```
Количество тревог: 5
```

Шаг 3. Оцениваем качество работы метода.

После получения тревог присоединяем известную истину для подсчёта ошибок.

Целевой сценарий — `long_session`: требуется обнаруживать внедрённое атакующее поведение, которое увеличивает длительность сеансов.

Качество детекции будем оценивать с помощью следующих метрик:

- количество верно обнаруженных атак;
- количество ложных срабатываний;
- количество пропущенных атак.

In [23]:
```
mad_unlabelled_scores_test = MAD_CONSTANT * (test["mean_session_min"].to_numpy()
mad_unlabelled_alerts_test = mad_unlabelled_scores_test > mad_unlabelled_thresho
mad_unlabelled_target_test = truth.loc[test.index, "attack_long_session"].to_num

# Каждое выражение сопоставляет тревогу и истину одного и того же часа.
mad_unlabelled_found = int((mad_unlabelled_alerts_test & mad_unlabelled_target_t
mad_unlabelled_false = int((mad_unlabelled_alerts_test & ~mad_unlabelled_target_
mad_unlabelled_missed = int((~mad_unlabelled_alerts_test & mad_unlabelled_target

mad_unlabelled_report = pd.Series({"Всего атак": int(mad_unlabelled_target_test.
                                   "Срабатываний": int(mad_unlabelled_alerts_tes
                                   "Правильно обнаружено": mad_unlabelled_found,
                                   "Ложно обнаружено": mad_unlabelled_false,
                                   "Пропущено": mad_unlabelled_missed},
                                   name = "Модифицированный Z-score — на неразме
mad_unlabelled_report.to_frame()
```

Out[23]:

	Модифицированный Z-score — на неразмеченной выборке
Всего атак	4
Срабатываний	5
Правильно обнаружено	3
Ложно обнаружено	2
Пропущено	1

Отобразим наблюдения, на которых детектор правильно обнаружил атаки, выдал ложную тревогу и пропустил атаку.

In [24]:
```
mad_unlabelled_events = test[["timestamp", 'mean_session_min']].copy()
mad_unlabelled_events["Оценка"] = mad_unlabelled_scores_test
mad_unlabelled_events["Целевой сценарий"] = mad_unlabelled_target_test
mad_unlabelled_events["Тревога"] = mad_unlabelled_alerts_test
mad_unlabelled_events["Сведения генератора"] = truth.loc[test.index, "legitimate
mad_unlabelled_target_examples = mad_unlabelled_events.loc[mad_unlabelled_target
mad_unlabelled_false_examples = mad_unlabelled_events.loc[mad_unlabelled_alerts_
display(pd.concat([mad_unlabelled_target_examples, mad_unlabelled_false_examples
```

	timestamp	mean_session_min	Оценка	Целевой сценарий	Тревога	Сведения генератора
1490	2026-03-04 02:00:00	47.0000	2.0455	True	False	внедрённый целевой атакующий сценарий
1522	2026-03-05 10:00:00	60.0000	4.2414	False	True	плановое длительное обслуживание
1650	2026-03-10 18:00:00	57.0000	3.7347	True	True	внедрённый целевой атакующий сценарий
1820	2026-03-17 20:00:00	75.0000	6.7752	True	True	внедрённый целевой атакующий сценарий
1920	2026-03-22 00:00:00	58.0000	3.9036	False	True	плановое длительное обслуживание
2050	2026-03-27 10:00:00	105.0000	11.8428	True	True	внедрённый целевой атакующий сценарий

- **Обычный Z-score на неразмеченной выборке**:

    - порог ≈ 83.35 минут;
    - из-за включения аномалий в расчёт σ порог завышен — детектируется только 1 из 4 атак, ложных срабатываний нет.
- **Модифицированный Z-score на неразмеченной выборке**:
    - порог ≈ 55.61 минут;
    - устойчив к выбросам — порог остаётся чувствительным даже при наличии аномалий в выборке.
    - результат: 3 из 4 атак найдено, 2 ложных срабатывания.

Вывод: модифицированный Z-score эффективнее на неразмеченных данных — за счёт устойчивости к выбросам; сохраняет высокий полноту обнаружения атак при умеренном количестве ложных срабатываний.

Модифицированный Z-score: сопоставление двух режимов

В обоих режимах использовано одно заранее заданное правило. Различия возникают из-за состава истории и оценённых по ней параметров. Сначала сопоставим параметры, затем фактические решения.

In [25]:
```
pd.DataFrame({"Выборка": ["Нормальная", "Неразмеченная"],
              "Медиана, мин.": [mad_median_normal, mad_median_unlabelled],
              "MAD, мин.": [mad_scale_normal, mad_scale_unlabelled],
              "Выбранная граница, мин.": [
        mad_median_normal + mad_normal_threshold * mad_scale_normal / MAD_CONSTA
        mad_median_unlabelled + mad_unlabelled_threshold * mad_scale_unlabelled
```

Out[25]:

	Выборка	Медиана, мин.	MAD, мин.	Выбранная граница, мин.
0	Нормальная	34.7615	3.9245	55.1262
1	Неразмеченная	34.8905	3.9930	55.6106

5.3. Межквартильный размах (IQR)

Межквартильный размах (IQR - Interquartile Range) — робастная мера разброса, основанная на квартилях:

$$\mathrm{IQR} = Q_3 - Q_1,$$

где:

- Q_1 — первый квартиль (25-й перцентиль);
- Q_3 — третий квартиль (75-й перцентиль).

Границы выбросов:

- нижняя граница: $Q_1 - 1{,}5 \cdot \mathrm{IQR}$;
- верхняя граница: $Q_3 + 1{,}5 \cdot \mathrm{IQR}$.

Экстремальные выбросы определяют по более жёстким границам:

- нижняя: $Q_1 - 3 \cdot \mathrm{IQR}$;
- верхняя: $Q_3 + 3 \cdot \mathrm{IQR}$.

В большинстве случаев требуется только верхний порог. Тогда общее правило принятия решений об аномальности объекта будет иметь следующий вид:

$$x > Q_3 + T \cdot \mathrm{IQR}.$$

Метод IQR не требует нормального распределения значений признака и не использует среднее. Небольшое число очень больших значений часто почти не меняет квартильные позиции. Однако **отсутствие требования нормальности не означает одинаковой пригодности для любых распределений**.

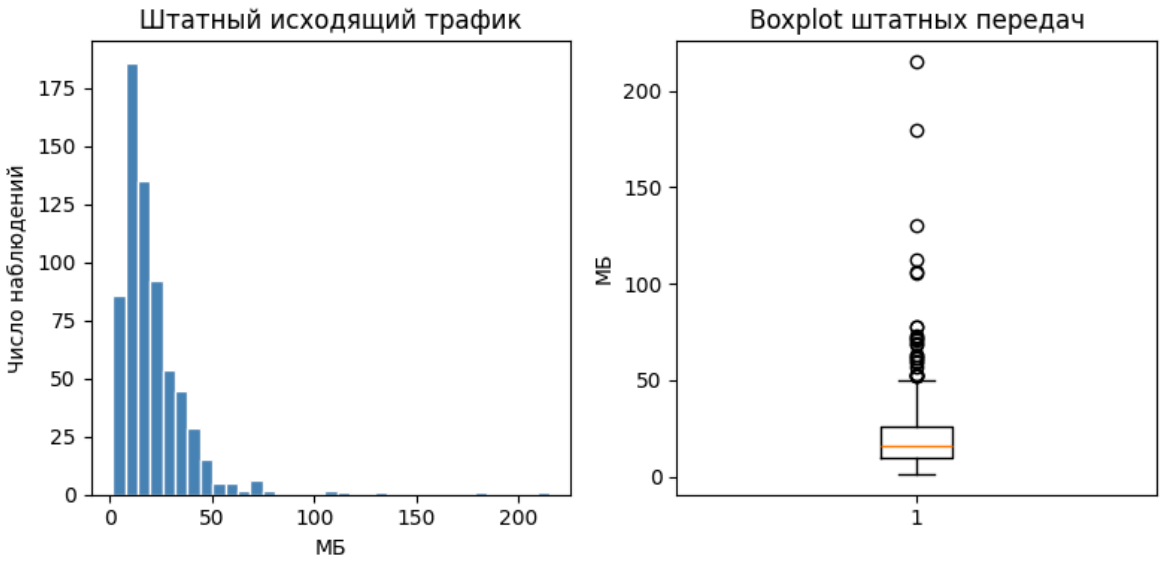
При длинном правом хвосте законные большие передачи могут систематически превышать верхнюю границу. При нескольких режимах общий диапазон может быть слишком широким для одного режима и слишком узким для другого. Значительное загрязнение способно сдвинуть сами квартили.

Рассмотрим пример использования метода для поиска аномалий в значениях признака `bytes_out_mb` (объем исходящего трафика).

IQR в режиме обучения только на норме

Визуальный анализ распределения признака.

In [26]:
```
fig, axes = plt.subplots(1, 2, figsize=(8, 4))
values = normal_train["bytes_out_mb"]
axes[0].hist(values, bins=35, color="steelblue", edgecolor="white")
axes[0].set(title="Штатный исходящий трафик", xlabel = "МБ", ylabel = "Число наб
axes[1].boxplot(values, whis=1.5)
axes[1].set(title = "Boxplot штатных передач", ylabel = "МБ")
plt.tight_layout()
```



Нормальный трафик без атак имеет длинный правый хвост: он соответствует наблюдениям с законными экспортами данных. Точки за «усами» `boxplot` на этой нормальной истории — потенциальные ложные тревоги стандартного правила, а не доказательство того, что история ошибочно размечена.

Шаг 1. Оцениваем параметры.

Используем `normal_train`.

In [27]:
```
iqr_q1_normal = normal_train["bytes_out_mb"].quantile(0.25)
iqr_q3_normal = normal_train["bytes_out_mb"].quantile(0.75)
iqr_width_normal = iqr_q3_normal - iqr_q1_normal

display(pd.Series({"Первый квартиль, МБ": iqr_q1_normal,
                   "Третий квартиль, МБ": iqr_q3_normal,
                   "IQR, МБ": iqr_width_normal,
                   "Нижняя граница при k=1.5": iqr_q1_normal - 1.5 * iqr_width_n
                   "Верхняя граница при k=1.5": iqr_q3_normal + 1.5 * iqr_width_

k = 1.5
iqr_normal_raw_boundary = iqr_q3_normal + k * iqr_width_normal
print("Граница в исходной шкале:", round(iqr_normal_raw_boundary, 2))
```

	Параметры
Первый квартиль, МБ	10.0802
Третий квартиль, МБ	26.0390
IQR, МБ	15.9588
Нижняя граница при k=1.5	-13.8579
Верхняя граница при k=1.5	49.9771

```
Граница в исходной шкале: 49.98
```

Шаг 2. Применяем заранее заданное правило к новым данным.

Для объектов тестовой выборки `test` сравниваем значения признака с вычисленной верхней границей.

In [28]:
```
iqr_normal_threshold = iqr_q3_normal + 1.5 * iqr_width_normal
iqr_normal_alerts_test = test["bytes_out_mb"].to_numpy() > iqr_normal_threshold
print(f'Количество тревог: {iqr_normal_alerts_test.sum()}')
```

```
Количество тревог: 38
```

Шаг 3. Оцениваем качество работы метода.

После получения тревог присоединяем известную истину для подсчёта ошибок.

Целевой сценарий — `exfiltration`: требуется обнаруживать внедрённое атакующее поведение, которое увеличивает объём передаваемого трафика.

Качество детекции будем оценивать с помощью следующих метрик:

- количество верно обнаруженных атак;
- количество ложных срабатываний;
- количество пропущенных атак.

In [29]:
```
iqr_normal_target_test = truth.loc[test.index, "attack_exfiltration"].to_numpy()

# Каждое выражение сопоставляет тревогу и истину одного и того же часа.
iqr_normal_found = int((iqr_normal_alerts_test & iqr_normal_target_test).sum())
iqr_normal_false = int((iqr_normal_alerts_test & ~iqr_normal_target_test).sum())
iqr_normal_missed = int((~iqr_normal_alerts_test & iqr_normal_target_test).sum()

iqr_normal_report = pd.Series({"Всего атак": int(iqr_normal_target_test.sum()),
                               "Срабатываний": int(iqr_normal_alerts_test.sum())
                               "Правильно обнаружено": iqr_normal_found,
                               "Ложно обнаружено": iqr_normal_false,
                               "Пропущено": iqr_normal_missed},
                               name = "IQR — на нормальной выборке")
iqr_normal_report.to_frame()
```

Out[29]:

	IQR — на нормальной выборке
Всего атак	4
Срабатываний	38
Правильно обнаружено	3
Ложно обнаружено	35
Пропущено	1

Отобразим наблюдения, на которых детектор правильно обнаружил атаки, выдал ложную тревогу и пропустил атаку.

In [30]:
```
# Отобразим первые 10 событий.
iqr_normal_events = test[["timestamp", 'bytes_out_mb']].copy()
iqr_normal_events["Целевой сценарий"] = iqr_normal_target_test
iqr_normal_events["Тревога"] = iqr_normal_alerts_test
iqr_normal_events["Сведения генератора"] = truth.loc[test.index, "legitimate_not
iqr_normal_target_examples = iqr_normal_events.loc[iqr_normal_target_test]
iqr_normal_false_examples = iqr_normal_events.loc[iqr_normal_alerts_test & ~iqr_
print("Граница в исходной шкале:", round(iqr_normal_raw_boundary, 2))
display(pd.concat([iqr_normal_target_examples, iqr_normal_false_examples]).sort_
```

```
Граница в исходной шкале: 49.98
```

	timestamp	bytes_out_mb	Целевой сценарий	Тревога	Сведения генератора
1449	2026-03-02 09:00:00	78.6830	False	True	обычная штатная работа
1478	2026-03-03 14:00:00	75.2310	False	True	обычная штатная работа
1487	2026-03-03 23:00:00	61.9110	False	True	обычная штатная работа
1489	2026-03-04 01:00:00	63.9950	False	True	обычная штатная работа
1518	2026-03-05 06:00:00	105.2590	False	True	обычная штатная работа
1525	2026-03-05 13:00:00	92.6540	False	True	обычная штатная работа
1530	2026-03-05 18:00:00	130.0000	False	True	разрешённый экспорт отчётов
1541	2026-03-06 05:00:00	54.4100	False	True	обычная штатная работа
1550	2026-03-06 14:00:00	40.0000	True	False	внедрённый целевой атакующий сценарий
1556	2026-03-06 20:00:00	55.2490	False	True	обычная штатная работа

При $k = 1.5$ метод чувствителен даже к умеренным отклонениям — в нормальной выборке часть «пограничных» значений интерпретируется как выбросы. Высокая ложных срабатываний указывает на недостаточную селективность порога: распределение признака содержит естественные вариации, близкие к аномальным.

При $k = 1.5$ IQR обеспечивает высокий долю выявления атак, но неприемлем из-за экстремально высокиого количества ложных срабатываний. Для их снижения требуется увеличение k (например, до 2.0–3.0) либо постфильтрация срабатываний.

Ограничения метода.

1. Правило 1,5*IQR является эвристикой, а не формальным доказательством аномальности.
2. Метод не учитывает временной и пользовательский контекст.
3. Для распределений с длинным правым хвостом может возникать много ложных срабатываний.
4. Один общий порог может быть неправильным для разных групп устройств. Например, нормальный трафик сервера и рабочей станции существенно различается.

IQR в режиме обучения на неразмеченной выборке

Шаг 1. Оцениваем параметры.

Используем смешанную обучающую выборку `train`, которая содержит как нормальные наблюдения, так и наблюдения с атакующими сценариями.

In [31]:
```
iqr_q1_unlabelled = train["bytes_out_mb"].quantile(0.25)
iqr_q3_unlabelled = train["bytes_out_mb"].quantile(0.75)
iqr_width_unlabelled = iqr_q3_unlabelled - iqr_q1_unlabelled

display(pd.Series({"Первый квартиль, МБ": iqr_q1_unlabelled,
                   "Третий квартиль, МБ": iqr_q3_unlabelled,
                   "IQR, МБ": iqr_width_unlabelled,
                   "Нижняя граница при k=1.5": iqr_q1_unlabelled - 1.5 * iqr_wid
                   "Верхняя граница при k=1.5": iqr_q3_unlabelled + 1.5 * iqr_wi

k = 1.5
iqr_unlabelled_raw_boundary = iqr_q3_unlabelled + k * iqr_width_unlabelled
print("Граница оценки:", iqr_unlabelled_raw_boundary)
print("Граница в исходной шкале:", iqr_unlabelled_raw_boundary)
```

	Параметры
Первый квартиль, МБ	10.1032
Третий квартиль, МБ	25.9587
IQR, МБ	15.8555
Нижняя граница при k=1.5	-13.6800
Верхняя граница при k=1.5	49.7420

```
Граница оценки: 49.742
Граница в исходной шкале: 49.742
```

Шаг 2. Применяем заранее заданное правило к новым данным.

Для объектов тестовой выборки `test` сравниваем значения признака с вычисленной верхней границей.

In [32]:
```
iqr_unlabelled_threshold = iqr_q3_unlabelled + 1.5 * iqr_width_unlabelled
iqr_unlabelled_alerts_test = test["bytes_out_mb"].to_numpy() > iqr_unlabelled_th
print(f'Количество тревог: {iqr_unlabelled_alerts_test.sum()}')
```

```
Количество тревог: 38
```

Шаг 3. Оцениваем качество работы метода.

После получения тревог присоединяем известную истину для подсчёта ошибок.

Целевой сценарий — `exfiltration`: требуется обнаруживать внедрённое атакующее поведение, которое увеличивает объём передаваемого трафика.

Качество детекции будем оценивать с помощью следующих метрик:

- количество верно обнаруженных атак;
- количество ложных срабатываний;

- количество пропущенных атак.

In [33]:
```
iqr_unlabelled_target_test = truth.loc[test.index, "attack_exfiltration"].to_num

iqr_unlabelled_found = int((iqr_unlabelled_alerts_test & iqr_unlabelled_target_t
iqr_unlabelled_false = int((iqr_unlabelled_alerts_test & ~iqr_unlabelled_target_
iqr_unlabelled_missed = int((~iqr_unlabelled_alerts_test & iqr_unlabelled_target

iqr_unlabelled_report = pd.Series({"Всего атак": int(iqr_unlabelled_target_test.
                                   "Срабатываний": int(iqr_unlabelled_alerts_tes
                                   "Правильно обнаружено": iqr_unlabelled_found,
                                   "Ложно обнаружено": iqr_unlabelled_false,
                                   "Пропущено": iqr_unlabelled_missed},
                                   name = "IQR — на неразмеченной выборке")
iqr_unlabelled_report.to_frame()
```

Out[33]:

	IQR — на неразмеченной выборке
Всего атак	4
Срабатываний	38
Правильно обнаружено	3
Ложно обнаружено	35
Пропущено	1

Отобразим наблюдения, на которых детектор правильно обнаружил атаки, выдал ложную тревогу и пропустил атаку.

In [34]:
```
iqr_unlabelled_events = test[["timestamp", 'bytes_out_mb']].copy()
iqr_unlabelled_events["Целевой сценарий"] = iqr_unlabelled_target_test
iqr_unlabelled_events["Тревога"] = iqr_unlabelled_alerts_test
iqr_unlabelled_events["Сведения генератора"] = truth.loc[test.index, "legitimate
iqr_unlabelled_target_examples = iqr_unlabelled_events.loc[iqr_unlabelled_target
iqr_unlabelled_false_examples = iqr_unlabelled_events.loc[iqr_unlabelled_alerts_
display(pd.concat([iqr_unlabelled_target_examples, iqr_unlabelled_false_examples
```

	timestamp	bytes_out_mb	Целевой сценарий	Тревога	Сведения генератора
1449	2026-03-02 09:00:00	78.6830	False	True	обычная штатная работа
1478	2026-03-03 14:00:00	75.2310	False	True	обычная штатная работа
1487	2026-03-03 23:00:00	61.9110	False	True	обычная штатная работа
1489	2026-03-04 01:00:00	63.9950	False	True	обычная штатная работа
1518	2026-03-05 06:00:00	105.2590	False	True	обычная штатная работа
1525	2026-03-05 13:00:00	92.6540	False	True	обычная штатная работа
1530	2026-03-05 18:00:00	130.0000	False	True	разрешённый экспорт отчётов
1541	2026-03-06 05:00:00	54.4100	False	True	обычная штатная работа
1550	2026-03-06 14:00:00	40.0000	True	False	внедрённый целевой атакующий сценарий
1556	2026-03-06 20:00:00	55.2490	False	True	обычная штатная работа

IQR: сопоставление двух режимов

В обоих режимах использовано одно заранее заданное правило. Различия возникают из-за состава выборок и оценённых по ней параметров. Сначала сопоставим параметры, затем фактические решения.

In [35]:
```
iqr_comparison = pd.concat([iqr_normal_report, iqr_unlabelled_report], axis = 1)
iqr_comparison
```

Out[35]:

	Всего атак	Срабатываний	Правильно обнаружено	Ложно обнаружено	Пропущено
IQR — на нормальной выборке	4	38	3	35	1
IQR — на неразмеченной выборке	4	38	3	35	1

In [36]:
```
pd.DataFrame({"Выборка": ["Нормальная", "Неразмеченная"],
              "Q1, МБ": [iqr_q1_normal, iqr_q1_unlabelled],
              "Q3, МБ": [iqr_q3_normal, iqr_q3_unlabelled],
              "IQR, МБ": [iqr_width_normal, iqr_width_unlabelled],
```

```
            "Выбранная верхняя граница, МБ": [
        iqr_q3_normal + k * iqr_width_normal,
        iqr_q3_unlabelled + k * iqr_width_unlabelled]})
```

Out[36]:

	Выборка	Q1, МБ	Q3, МБ	IQR, МБ	Выбранная верхняя граница, МБ
0	Нормальная	10.0802	26.0390	15.9588	49.9771
1	Неразмеченная	10.1032	25.9587	15.8555	49.7420

Полученный результат подтверждает устойчивость IQR: даже если в выборку попали аномалии, они не изменяют квартили и не сильно сдвигают границу.

IQR можно использовать на неразмеченных данных и получать почти такой же порог, как на чистой норме. Это удобно, если нет надёжной разметки.

В обоих режимах множитель равен 1,5. На длинном правом хвосте штатного трафика это правило может дать много ложных тревог. Сохранение квартилей при загрязнении не доказывает пригодности границы для конкретного распределения. Законный экспорт может быть больше умеренной утечки; одним объёмом невозможно всегда разделить такие события. Нужны сведения об адресате, разрешениях или профиле операций.

5.4. Квантильные пороги

Квантиль $(1-\alpha)$-уровня $Q_{(1-\alpha)}$ — это значение, **ниже** которого находится $(1-\alpha)\cdot 100\%$ наблюдений.

Тогда $\alpha \cdot 100\%$ наблюдений лежат **выше** этого значения. Т. е. α — это доля самых больших (или самых экстремальных) значений признака, которые мы «отрезаем» справа.

Например:

- $Q_{(1-0{,}05)}$ — 95-й процентиль;
- $Q_{(1-0{,}01)}$ — 99-й процентиль;
- $Q_{(1-0{,}005)}$ — 99,5-й процентиль.

Для выявления аномально больших значений можно использовать правило:

$$x_i > Q_{(1-\alpha)} \Longrightarrow \text{тревога}.$$

Для двустороннего анализа применяют условие:

$$x_i < Q_{\alpha/2} \quad \text{или} \quad x_i > Q_{1-\alpha/2}.$$

Квантильные пороги полезны, когда:

- неизвестна форма распределения;
- необходимо контролировать долю формируемых предупреждений;
- данные сильно асимметричны;

- требуется простой порог для промышленной системы мониторинга.

Рассмотрим пример использования метода для поиска аномалий в значениях признака `bytes_out_mb` (объем исходящего трафика).

Квантильный порог в режиме обучения только на норме

Шаг 1. Оцениваем параметры.

Используем выборку `normal_train` .

Для построения порогов будем использовать квантиль распределения признака на уровне 0,995.

In [37]:
```
quantile_values_normal = normal_train["bytes_out_mb"].to_numpy(dtype = float)
display(pd.Series(quantile_values_normal).describe().to_frame("Обучающий трафик,

quantile_normal_threshold = np.quantile(quantile_values_normal, 0.995)
print("Граница в исходной шкале:", round(quantile_normal_threshold, 2))
```

	Обучающий трафик, МБ
count	668.0000
mean	20.7506
std	17.8504
min	1.4330
25%	10.0802
50%	16.2045
75%	26.0390
max	215.0240

```
Граница в исходной шкале: 110.57
```

Шаг 2. Применяем заранее заданное правило к новым данным.

Для объектов тестовой выборки `test` сравниваем значения признака с вычисленной верхней границей.

In [38]:
```
quantile_normal_scores_test = test["bytes_out_mb"].to_numpy()
quantile_normal_alerts_test = quantile_normal_scores_test > quantile_normal_thre
print(f'Количество тревог: {quantile_normal_alerts_test.sum()}')
```

```
Количество тревог: 4
```

Шаг 3. Оцениваем качество работы метода.

После получения тревог присоединяем известную истину для подсчёта ошибок.

Целевой сценарий — `exfiltration` : требуется обнаруживать внедрённое атакующее поведение, которое увеличивает объём передаваемого трафика.

Качество детекции будем оценивать с помощью следующих метрик:

- количество верно обнаруженных атак;
- количество ложных срабатываний;
- количество пропущенных атак.

In [39]:
```
quantile_normal_target_test = truth.loc[test.index, "attack_exfiltration"].to_nu

quantile_normal_found = int((quantile_normal_alerts_test & quantile_normal_targe
quantile_normal_false = int((quantile_normal_alerts_test & ~quantile_normal_targ
quantile_normal_missed = int((~quantile_normal_alerts_test & quantile_normal_tar

quantile_normal_report = pd.Series({"Всего атак": int(quantile_normal_target_tes
                                    "Срабатываний": int(quantile_normal_alerts_t
                                    "Правильно обнаружено": quantile_normal_foun
                                    "Ложно обнаружено": quantile_normal_false,
                                    "Пропущено": quantile_normal_missed},
                                    name = "Квантильный порог — на нормальной вы
quantile_normal_report.to_frame()
```

Out[39]:

	Квантильный порог — на нормальной выборке
Всего атак	4
Срабатываний	4
Правильно обнаружено	2
Ложно обнаружено	2
Пропущено	2

Отобразим наблюдения, на которых детектор правильно обнаружил атаки, выдал ложную тревогу и пропустил атаку.

In [40]:
```
quantile_normal_events = test[["timestamp", 'bytes_out_mb']].copy()
quantile_normal_events["Целевой сценарий"] = quantile_normal_target_test
quantile_normal_events["Тревога"] = quantile_normal_alerts_test
quantile_normal_events["Сведения генератора"] = truth.loc[test.index, "legitimat
quantile_normal_target_examples = quantile_normal_events.loc[quantile_normal_tar
quantile_normal_false_examples = quantile_normal_events.loc[quantile_normal_aler
print("Граница в исходной шкале:", round(quantile_normal_threshold, 2))
pd.concat([quantile_normal_target_examples, quantile_normal_false_examples]).sor
```

```
Граница в исходной шкале: 110.57
```

Out[40]:

	timestamp	bytes_out_mb	Целевой сценарий	Тревога	Сведения генератора
1530	2026-03-05 18:00:00	130.0000	False	True	разрешённый экспорт отчётов
1550	2026-03-06 14:00:00	40.0000	True	False	внедрённый целевой атакующий сценарий
1700	2026-03-12 20:00:00	70.0000	True	False	внедрённый целевой атакующий сценарий
1880	2026-03-20 08:00:00	240.0000	True	True	внедрённый целевой атакующий сценарий
1940	2026-03-22 20:00:00	180.0000	False	True	разрешённый экспорт отчётов
2090	2026-03-29 02:00:00	600.0000	True	True	внедрённый целевой атакующий сценарий

Квантильный порог в режиме обучения на неразмеченной выборке

Шаг 1. Оцениваем параметры.

Используем смешанную обучающую выборку `train`, которая содержит как нормальные наблюдения, так и наблюдения с атакующими сценариями.

In [41]:
```
# Обучающая часть квантильного детектора — набор наблюдавшихся объёмов,
# то есть эмпирическое распределение.
quantile_values_unlabelled = train["bytes_out_mb"].to_numpy(dtype = float)
display(pd.Series(quantile_values_unlabelled).describe().to_frame("Обучающий тра

quantile_unlabelled_threshold = np.quantile(quantile_values_unlabelled, 0.995)
print("Граница в исходной шкале:", round(quantile_unlabelled_threshold, 2))
```

	Обучающий трафик, МБ
count	720.0000
mean	22.3455
std	33.8465
min	1.4330
25%	10.1032
50%	16.3525
75%	25.9587
max	680.0000

```
Граница в исходной шкале: 150.25
```

Шаг 2. Применяем заранее заданное правило к новым данным.

Для объектов тестовой выборки `test` сравниваем значения признака с вычисленной верхней границей.

In [42]:
```
quantile_unlabelled_scores_test = test["bytes_out_mb"].to_numpy()
quantile_unlabelled_alerts_test = quantile_unlabelled_scores_test > quantile_unl
print(f'Количество тревог: {quantile_unlabelled_alerts_test.sum()}')
```

```
Количество тревог: 3
```

Шаг 3. Оцениваем качество работы метода.

После получения тревог присоединяем известную истину для подсчёта ошибок.

Целевой сценарий — `exfiltration`: требуется обнаруживать внедрённое атакующее поведение, которое увеличивает объём передаваемого трафика.

Качество детекции будем оценивать с помощью следующих метрик:

- количество верно обнаруженных атак;
- количество ложных срабатываний;
- количество пропущенных атак.

In [43]:
```
quantile_unlabelled_target_test = truth.loc[test.index, "attack_exfiltration"].t

quantile_unlabelled_found = int((quantile_unlabelled_alerts_test & quantile_unla
quantile_unlabelled_false = int((quantile_unlabelled_alerts_test & ~quantile_unl
quantile_unlabelled_missed = int((~quantile_unlabelled_alerts_test & quantile_un

quantile_unlabelled_report = pd.Series({"Всего атак": int(quantile_unlabelled_ta
                                        "Срабатываний": int(quantile_unlabelled_
                                        "Правильно обнаружено": quantile_unlabel
                                        "Ложно обнаружено": quantile_unlabelled_
                                        "Пропущено": quantile_unlabelled_missed}
                                        name = "Квантильный порог — на неразмече
quantile_unlabelled_report.to_frame()
```

Out[43]:

	Квантильный порог — на неразмеченной выборке
Всего атак	4
Срабатываний	3
Правильно обнаружено	2
Ложно обнаружено	1
Пропущено	2

Отобразим наблюдения, на которых детектор правильно обнаружил атаки, выдал ложную тревогу и пропустил атаку.

In [44]:
```
quantile_unlabelled_events = test[["timestamp", 'bytes_out_mb']].copy()
quantile_unlabelled_events["Целевой сценарий"] = quantile_unlabelled_target_test
quantile_unlabelled_events["Тревога"] = quantile_unlabelled_alerts_test
quantile_unlabelled_events["Сведения генератора"] = truth.loc[test.index, "legit
quantile_unlabelled_target_examples = quantile_unlabelled_events.loc[quantile_un
```

```
quantile_unlabelled_false_examples = quantile_unlabelled_events.loc[quantile_unl
print("Граница в исходной шкале:", round(quantile_unlabelled_threshold, 2))
display(pd.concat([quantile_unlabelled_target_examples, quantile_unlabelled_fals
```

```
Граница в исходной шкале: 150.25
```

	timestamp	bytes_out_mb	Целевой сценарий	Тревога	Сведения генератора
1550	2026-03-06 14:00:00	40.0000	True	False	внедрённый целевой атакующий сценарий
1700	2026-03-12 20:00:00	70.0000	True	False	внедрённый целевой атакующий сценарий
1880	2026-03-20 08:00:00	240.0000	True	True	внедрённый целевой атакующий сценарий
1940	2026-03-22 20:00:00	180.0000	False	True	разрешённый экспорт отчётов
2090	2026-03-29 02:00:00	600.0000	True	True	внедрённый целевой атакующий сценарий

Квантильный порог: сопоставление двух режимов

В обоих режимах использовано одно заранее заданное правило. Различия возникают из-за состава истории и оценённых по ней параметров. Сначала сопоставим параметры, затем фактические решения.

In [45]:
```
quantile_comparison = pd.concat([quantile_normal_report, quantile_unlabelled_rep
quantile_comparison
```

Out[45]:

	Всего атак	Срабатываний	Правильно обнаружено	Ложно обнаружено	Пропущено
Квантильный порог — на нормальной выборке	4	4	2	2	2
Квантильный порог — на неразмеченной выборке	4	3	2	1	2

In [46]:
```
pd.DataFrame({"Выборка": ["Нормальная", "Неразмеченная"],
              "Выбранная верхняя граница (Q_0.995), МБ": [quantile_normal_thresh
                                                          quantile_unlabelled_th
```

Out[46]:

	Выборка	Выбранная верхняя граница (Q_0.995), МБ
0	Нормальная	110.5722
1	Неразмеченная	150.2500

Квантиль $0{,}995$ — это значение, правее которого лежит $0{,}5\%$ наблюдений. Если в выборку попадают аномально высокие значения (атаки или всплески трафика), они сдвигают правый хвост распределения и сам квантиль вправо.

- В **нормальной выборке** хвост «чистый»: квантиль $0{,}995 = 110,5722$.
- В **неразмеченной выборке** в хвосте уже есть аномальные значения — чтобы захватить те же $0{,}5\%$ самых больших наблюдений, нужно подняться выше: квантиль $0{,}995 = 150,25$.
- **Если цель — уменьшить ложные срабатывания**, квантильный порог на неразмеченной выборке может быть даже предпочтительнее: он автоматически становится строже и уменьшает число срабатываний.
- **Но он пропускает те же атаки**, что и порог на норме. Если эти 2 пропущенные атаки — критические, такой порог недостаточен.
- **Риск «перестраховки»**: при большом числе аномалий порог может уйти ещё выше и перестать замечать даже серьёзные всплески.

Ограничения метода:

1. Квантильный порог гарантирует статистическую редкость, но не подтверждает атаку.
2. На маленькой выборке оценки крайних квантилей нестабильны.
3. Если порог рассчитывается на данных, уже содержащих атаки, он может быть завышен.
4. При изменении поведения системы порог необходимо пересчитывать.
5. Фиксированная доля предупреждений может быть неудобна. Система будет формировать предупреждения даже при отсутствии реальных атак.
6. В промышленной системе порог нужно определять на отдельном историческом интервале, а затем применять к новым данным.

5.5. Обнаружение аномалий в счётных данных на основе распределения Пуассона

Распределение Пуассона используется для описания количества событий за заданный интервал времени. На его основе можно обнаруживать необычно большие значения счётчиков: число неудачных попыток входа за час, ошибок приложения за минуту или обращений к определённому ресурсу за пять минут. Признак должен принимать неотрицательные целые значения: 0, 1, 2 и так далее.

Условия применения. Сравниваемые интервалы должны иметь одинаковую длительность, а данные — относиться к достаточно однородному режиму работы. Предполагается, что события возникают примерно независимо, а их обычная интенсивность остаётся примерно постоянной. Например, если дневная и ночная

активность существенно различаются, для них могут потребоваться отдельные модели.

В распределении Пуассона среднее и дисперсия равны. Поэтому заметное превышение выборочной дисперсии над средним служит поводом проверить применимость модели: причиной могут быть связанные всплески событий, смешение разных режимов или загрязнение истории. Одного соответствия целочисленному формату недостаточно.

Логика обнаружения. По обучающей истории оценивают параметр λ — среднее количество событий за один интервал. Затем по распределению Пуассона рассчитывают верхнюю границу: значения выше неё должны редко встречаться в принятой модели штатного поведения. Если новый счётчик превышает эту границу, детектор формирует тревогу.

Распределение Пуассона описывает число событий в фиксированном интервале при простой модели независимого появления с постоянной средней интенсивностью. Его функция вероятностей:

$$P(X = k) = \frac{\lambda^k \cdot e^{-\lambda}}{k!}, \qquad k = 0, 1, 2, \ldots.$$

Здесь X — случайное число событий за интервал; k — конкретное неотрицательное целое число событий; $P(X = k)$ — вероятность такого числа; $\lambda \geq 0$ — ожидаемое число событий за выбранный интервал.

Рассмотрим технологию формирования детектора аномалий с использованием модели Пуассона на примере признака `failed_logins` - количество неудачных попыток входа за час. Необычно большой счётчик может указывать на подбор пароля, но также возникать из-за ошибочной настройки приложения или законных действий пользователя. Поэтому превышение границы означает статистическое отклонение, требующее проверки, а не подтверждённую атаку.

Естественная оценка среднего числа неудачных попыток за один час:

$$\widehat{\lambda} = \frac{1}{n} \sum_{i=1}^{n} x_i,$$

где:

- x_i — количество неудачных попыток за i-й одинаковый интервал (один час);
- n — число таких интервалов;
- $\widehat{\lambda}$ — оценённый параметр. На норме — это оценка штатного среднего. На неразмеченной истории сильные всплески могут её повысить, расширив допустимый диапазон.

Задаётся небольшая модельная вероятность верхнего хвоста α. Тогда граница определяется как:

$$u_\alpha = Q_{\mathrm{Pois}(\widehat{\lambda})}(1 - \alpha), \qquad x > u_\alpha \Longrightarrow \text{тревога},$$

где:

- $Q_{\mathrm{Pois}(\widehat{\lambda})}$ — квантиль распределения Пуассона с оценённым средним;
- u_α — целочисленная верхняя граница;
- α — заданная допустимая вероятность превышения этой границы в модели;
- x — количество неудачных попыток для нового объекта.

Смысл правила такой: **выбираем границу счётчика, превышение которой достаточно редко встречается в модели нормальной работы.**

Нпример, пусть обычное среднее количество неудачных попыток входа составляет **1,2 ошибки входа за час**, а мы задали $\alpha = 0{,}01$. Это означает, что мы хотим, чтобы по модели выше границы находилось **не более 1 % штатных событий**.

Для такого распределения Пуассона получаются следующие значения:

Граница u	Когда возникает тревога	Вероятность по модели
3	При 4 и более ошибках	Около 3,38 %
4	При 5 и более ошибках	Около 0,775 %

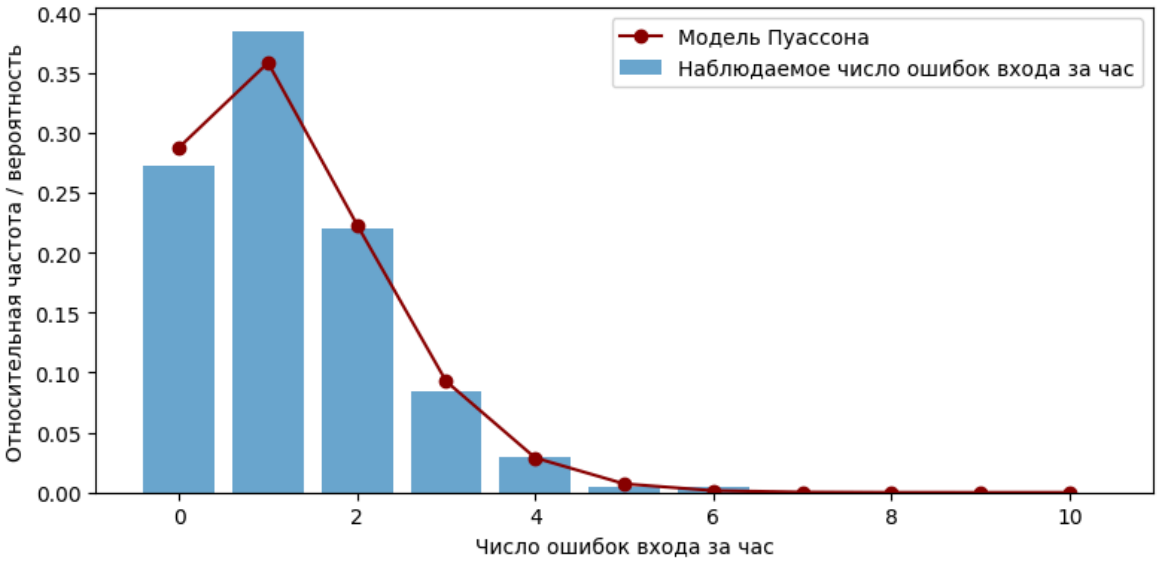
На реальных данных ложных тревог может оказаться больше. В действительности среднее может быть оценено неточно, нагрузка — измениться, а ошибки входа — возникать связанными всплесками. Тогда модель описывает штатное поведение недостаточно точно.

Поэтому $\alpha = 0{,}01$ означает **настройку модельной границы**, а не обещание ровно одной ложной тревоги на каждые сто часов. Реальные ложные срабатывания нужно считать на отдельных проверенных данных.

Модель Пуассона в режиме обучения только на норме

Визуальный анализ распределения признака.

In [47]:
```
# Частоты и вероятности сравниваются только для нормальной истории.
observed_login_frequencies = normal_train["failed_logins"].value_counts(normaliz
login_grid = np.arange(0, max(10, int(observed_login_frequencies.index.max())) +
plt.figure(figsize=(8, 4))
plt.bar(login_grid, observed_login_frequencies.reindex(login_grid, fill_value =
        alpha = 0.65, label="Наблюдаемое число ошибок входа за час")
plt.plot(login_grid, stats.poisson.pmf(login_grid, mu = normal_train["failed_log
         marker = "o", color = "darkred", label = "Модель Пуассона")
plt.xlabel("Число ошибок входа за час")
plt.ylabel("Относительная частота / вероятность")
plt.legend()
plt.tight_layout()
```



Основная масса наблюдаемых частот согласуется выбранной моделью Пуассона.

Шаг 1. Оцениваем параметры.

Используем выборку `normal_train`.

Допустимая вероятность превышения порога аномальности $\alpha = 0,01$.

In [48]:
```
poisson_values_normal = normal_train["failed_logins"].to_numpy(dtype = float)

# Оценка lambda — обычное обучающее среднее числа ошибок за час.
poisson_lambda_normal = poisson_values_normal.mean()
poisson_variance_normal = poisson_values_normal.var(ddof = 1)
display(pd.Series({"Среднее ошибок за час": poisson_lambda_normal,
                   "Дисперсия": poisson_variance_normal,
                   "Дисперсия / среднее": (poisson_variance_normal / poisson_lam

poisson_normal_threshold = stats.poisson.ppf(1 - 0.01, mu = poisson_lambda_norma
print("Граница в исходной шкале:", poisson_normal_threshold)
```

	Параметры
Среднее ошибок за час	1.2455
Дисперсия	1.2230
Дисперсия / среднее	0.9819

```
Граница в исходной шкале: 4.0
```

Оценённое среднее число ошибок за час равно 1.2455; дисперсия — 1.2230. Для идеального пуассоновского закона они равны, но выборочные оценки могут различаться. Большое расхождение требует исследования состава истории, а не автоматического вывода об атаке.

Шаг 2. Применяем заранее заданное правило к новым данным.

Для объектов тестовой выборки `test` сравниваем значения признака с вычисленной верхней границей.

In [49]:
```
poisson_normal_scores_test = test["failed_logins"].to_numpy()
poisson_normal_alerts_test = poisson_normal_scores_test > poisson_normal_thresho
print(f'Количество тревог: {poisson_normal_alerts_test.sum()}')
```

```
Количество тревог: 11
```

Шаг 3. Оцениваем качество работы метода.

После получения тревог присоединяем известную истину для подсчёта ошибок.

Целевой сценарий — `brute_force`: требуется обнаруживать внедрённое атакующее поведение, которое увеличивает число неудачных входов за час.

Качество детекции будем оценивать с помощью следующих метрик:

- количество верно обнаруженных атак;
- количество ложных срабатываний;
- количество пропущенных атак.

In [50]:
```
poisson_normal_target_test = truth.loc[test.index, "attack_brute_force"].to_nump

poisson_normal_found = int((poisson_normal_alerts_test & poisson_normal_target_t
poisson_normal_false = int((poisson_normal_alerts_test & ~poisson_normal_target_
poisson_normal_missed = int((~poisson_normal_alerts_test & poisson_normal_target

poisson_normal_report = pd.Series({"Всего атак": int(poisson_normal_target_test.
                                   "Срабатываний": int(poisson_normal_alerts_tes
                                   "Правильно обнаружено": poisson_normal_found,
                                   "Ложно обнаружено": poisson_normal_false,
                                   "Пропущено": poisson_normal_missed},
                                   name = "Модель Пуассона — на нормальной выбор
poisson_normal_report.to_frame()
```

Out[50]:

	Модель Пуассона — на нормальной выборке
Всего атак	4
Срабатываний	11
Правильно обнаружено	4
Ложно обнаружено	7
Пропущено	0

Отобразим наблюдения, на которых детектор правильно обнаружил атаки, выдал ложную тревогу и пропустил атаку.

In [51]:
```
poisson_normal_events = test[["timestamp", 'failed_logins']].copy()
poisson_normal_events["Целевой сценарий"] = poisson_normal_target_test
poisson_normal_events["Тревога"] = poisson_normal_alerts_test
poisson_normal_events["Сведения генератора"] = truth.loc[test.index, "legitimate
poisson_normal_target_examples = poisson_normal_events.loc[poisson_normal_target
poisson_normal_false_examples = poisson_normal_events.loc[poisson_normal_alerts_
print("Граница в исходной шкале:", poisson_normal_threshold)
display(pd.concat([poisson_normal_target_examples, poisson_normal_false_examples
```

```
Граница в исходной шкале: 4.0
```

	timestamp	failed_logins	Целевой сценарий	Тревога	Сведения генератора
1475	2026-03-03 11:00:00	8	True	True	внедрённый целевой атакующий сценарий
1535	2026-03-05 23:00:00	5	False	True	законные ошибки входа
1620	2026-03-09 12:00:00	11	True	True	внедрённый целевой атакующий сценарий
1674	2026-03-11 18:00:00	5	False	True	обычная штатная работа
1722	2026-03-13 18:00:00	6	False	True	внедрённый целевой атакующий сценарий
1800	2026-03-17 00:00:00	5	True	True	внедрённый целевой атакующий сценарий
1882	2026-03-20 10:00:00	6	False	True	обычная штатная работа
1950	2026-03-23 06:00:00	6	False	True	законные ошибки входа
2040	2026-03-27 00:00:00	20	True	True	внедрённый целевой атакующий сценарий
2120	2026-03-30 08:00:00	5	False	True	обычная штатная работа
2124	2026-03-30 12:00:00	6	False	True	обычная штатная работа

Модель Пуассона в режиме обучения на неразмеченной выборке

Шаг 1. Оцениваем параметры.

Используем смешанную обучающую выборку `train`, которая содержит как нормальные наблюдения, так и наблюдения с атакующими сценариями.

In [52]:
```
poisson_values_unlabelled = train["failed_logins"].to_numpy(dtype = float)

# Оценка lambda — обычное обучающее среднее числа ошибок за час.
poisson_lambda_unlabelled = poisson_values_unlabelled.mean()
poisson_variance_unlabelled = poisson_values_unlabelled.var(ddof=1)
display(pd.Series({"Среднее ошибок за час": poisson_lambda_unlabelled,
                   "Дисперсия": poisson_variance_unlabelled,
                   "Дисперсия / среднее": (poisson_variance_unlabelled / poisson

poisson_unlabelled_threshold = stats.poisson.ppf(1 - 0.01, mu = poisson_lambda_u
print("Граница в исходной шкале:", poisson_unlabelled_threshold)
```

	Параметры
Среднее ошибок за час	1.3097
Дисперсия	2.1362
Дисперсия / среднее	1.6310

```
Граница в исходной шкале: 5.0
```

Шаг 2. Применяем заранее заданное правило к новым данным.

Для объектов тестовой выборки `test` сравниваем значения признака с вычисленной верхней границей.

In [53]:
```
poisson_unlabelled_scores_test = test["failed_logins"].to_numpy()
poisson_unlabelled_alerts_test = poisson_unlabelled_scores_test > poisson_unlabe
print(f'Количество тревог: {poisson_unlabelled_alerts_test.sum()}')
```

```
Количество тревог: 7
```

Шаг 3. Оцениваем качество работы метода.

После получения тревог присоединяем известную истину для подсчёта ошибок.

Целевой сценарий — `brute_force`: требуется обнаруживать внедрённое атакующее поведение, которое увеличивает число неудачных входов за час.

Качество детекции будем оценивать с помощью следующих метрик:

- количество верно обнаруженных атак;
- количество ложных срабатываний;
- количество пропущенных атак.

In [54]:
```
poisson_unlabelled_target_test = truth.loc[test.index, "attack_brute_force"].to_

poisson_unlabelled_found = int((poisson_unlabelled_alerts_test & poisson_unlabel
poisson_unlabelled_false = int((poisson_unlabelled_alerts_test & ~poisson_unlabe
poisson_unlabelled_missed = int((~poisson_unlabelled_alerts_test & poisson_unlab

poisson_unlabelled_report = pd.Series({"Всего атак": int(poisson_unlabelled_targ
                                       "Срабатываний": int(poisson_unlabelled_al
                                       "Правильно обнаружено": poisson_unlabelle
                                       "Ложно обнаружено": poisson_unlabelled_fa
                                       "Пропущено": poisson_unlabelled_missed},
                                       name = "Модель Пуассона — на неразмеченно
poisson_unlabelled_report.to_frame()
```

Out[54]:

Модель Пуассона — на неразмеченной выборке	
Всего атак	4
Срабатываний	7
Правильно обнаружено	3
Ложно обнаружено	4
Пропущено	1

Отобразим наблюдения, на которых детектор правильно обнаружил атаки, выдал ложную тревогу и пропустил атаку.

In [55]:
```
poisson_unlabelled_events = test[["timestamp", 'failed_logins']].copy()
poisson_unlabelled_events["Целевой сценарий"] = poisson_unlabelled_target_test
poisson_unlabelled_events["Тревога"] = poisson_unlabelled_alerts_test
poisson_unlabelled_events["Сведения генератора"] = truth.loc[test.index, "legiti
poisson_unlabelled_target_examples = poisson_unlabelled_events.loc[poisson_unlab
poisson_unlabelled_false_examples = poisson_unlabelled_events.loc[poisson_unlabe
display(pd.concat([poisson_unlabelled_target_examples, poisson_unlabelled_false_
```

	timestamp	failed_logins	Целевой сценарий	Тревога	Сведения генератора
1475	2026-03-03 11:00:00	8	True	True	внедрённый целевой атакующий сценарий
1620	2026-03-09 12:00:00	11	True	True	внедрённый целевой атакующий сценарий
1722	2026-03-13 18:00:00	6	False	True	внедрённый целевой атакующий сценарий
1800	2026-03-17 00:00:00	5	True	False	внедрённый целевой атакующий сценарий
1882	2026-03-20 10:00:00	6	False	True	обычная штатная работа
1950	2026-03-23 06:00:00	6	False	True	законные ошибки входа
2040	2026-03-27 00:00:00	20	True	True	внедрённый целевой атакующий сценарий
2124	2026-03-30 12:00:00	6	False	True	обычная штатная работа

Модель Пуассона: сопоставление двух режимов

В обоих режимах использовано одно заранее заданное правило. Различия возникают из-за состава истории и оценённых по ней параметров. Сначала сопоставим параметры, затем фактические решения.

In [56]:
```
poisson_comparison = pd.concat([poisson_normal_report, poisson_unlabelled_report
poisson_comparison
```

Out[56]:

	Всего атак	Срабатываний	Правильно обнаружено	Ложно обнаружено	Пропущено
Модель Пуассона — на нормальной выборке	4	11	4	7	0
Модель Пуассона — на неразмеченной выборке	4	7	3	4	1

Модель Пуассона с уровнем значимости $\alpha = 0.01$ позволяет формально контролировать долю ложных срабатываний среди нормальных часов. Однако её порог чувствителен к составу обучающей выборки: при наличии аномалий в данных порог автоматически повышается, снижая число ложных срабатываний. На нормальной выборке модель достигает полного обнаружения атак ценой высокой доли ложных срабатываний; на неразмеченной — становится строже, снижая нагрузку на аналитика, но пропускает часть инцидентов. Это делает Пуассон удобным как базовый фильтр, но требует дополнительной приоритизации пограничных случаев.

5.6. Расстояние Махаланобиса

Рассмотрим два признака:

- `mean_connections_per_min` (среднее количество соединений в минуту);
- `mean_unique_destinations` (среднее количество уникальных адресатов).

При штатной работе среднее число адресатов обычно растёт с числом соединений. Злоумышленник может создать много соединений с небольшим числом узлов. Каждый признак отдельно не обязательно будет экстремальным, но их сочетание нарушит обычную зависимость.

Расстояние Махаланобиса учитывает:

- средние значения признаков;
- разброс каждого признака;
- корреляции между признаками.

Расстояние Махаланобиса относится к группе многомерных статистических методов обнаружения аномалий (multivariate statistical anomaly detection). Более узко — это параметрический метод на основе модели многомерного нормального распределения:

- многомерность: в отличие от Z-score или IQR, которые работают по одному признаку, расстояние Махаланобиса оценивает наблюдение сразу по набору признаков;
- учёт корреляций: используется ковариационную матрица, поэтому метод «видит» совместные аномалии;
- параметричность: метод опирается на предположение, что «норма» имеет многомерное нормальное распределение.

Пусть каждое наблюдение имеет m признаков и представлено вектором x_i. Для двух признаков:

$$x_i = \begin{pmatrix} a_i \\ b_i \end{pmatrix}, \qquad \widehat{\mu} = \begin{pmatrix} \bar{a} \\ \bar{b} \end{pmatrix},$$

где:

- i — номер наблюдения;
- a_i — среднее число соединений в минуту;
- b_i — среднее число уникальных адресатов;
- $\bar{a}$ и $\bar{b}$ — их выборочные средние;
- $\widehat{\mu}$ — оценка вектора центра.

Выборочная ковариация двух признаков:

$$\widehat{\mathrm{Cov}}(a,b) = \frac{1}{n-1}\sum_{i=1}^{n}(a_i - \bar{a})(b_i - \bar{b}).$$

Положительная ковариация означает, что отклонения обоих признаков от своих средних часто имеют одинаковый знак. Отрицательная — что знаки часто различаются. Ковариация сама по себе не доказывает причинную связь.

Ковариационная матрица — это квадратная матрица, которая описывает, как признаки в данных варьируются совместно.

Ковариационная матрица для двух признаков имеет вид::

$$\widehat{\Sigma} = \begin{pmatrix} \widehat{\mathrm{Var}}(a) & \widehat{\mathrm{Cov}}(a,b) \\ \widehat{\mathrm{Cov}}(b,a) & \widehat{\mathrm{Var}}(b) \end{pmatrix}.$$

Здесь $\widehat{\mathrm{Var}}$ — выборочная дисперсия; диагональные элементы описывают разброс каждого признака, внедиагональные — совместную изменчивость. Матрица симметрична. Ковариация признака с самим собой равна его дисперсии.

Формула расстояния Махаланобиса:

$$D(x) = \sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)},$$

где:

- x — вектор признаков;

- μ — вектор средних значений (центр «нормы»);
- Σ — ковариационная матрица признаков;
- Σ^{-1} — обратная ковариационная матрица.

При известных истинных параметрах и многомерном нормальном распределении квадрат расстояния имеет распределение χ^2 с k степенями свободы. Степени свободы здесь соответствуют числу признаков. Отправной порог можно записать как

$$D^2(x) > \chi^2_{k,q},$$

где $\chi^2_{k,q}$ — квантиль уровня q распределения χ^2 с k степенями свободы. Например, $q = 0{,}99$, а вероятность верхнего хвоста модели равна $\alpha = 1 - q = 0{,}01$.

Геометрически граница постоянного D^2 в двух измерениях является эллипсом. Его наклон отражает связь признаков, а размеры — масштабы изменчивости. Точка может быть близка к центру по обычному расстоянию, но оказаться далеко поперёк узкой вытянутой области нормы.

Расстояние Махаланобиса в режиме обучения только на норме

Шаг 1. Оцениваем параметры.

Используем выборку `normal_train`.

Для построения порогов будем использовать квантиль распределения признака на уровне 0,99.

In [57]:
```
JOINT_FEATURES = ["mean_connections_per_min", "mean_unique_destinations"]
maha_values_normal = normal_train[JOINT_FEATURES].to_numpy(dtype = float)
maha_center_normal = maha_values_normal.mean(axis = 0)
maha_covariance_normal = np.cov(maha_values_normal, rowvar = False, ddof = 1)

display(pd.Series(maha_center_normal, index = JOINT_FEATURES, name = "Центр").to
display(pd.DataFrame(maha_covariance_normal, index = JOINT_FEATURES, columns = J

maha_normal_threshold = stats.chi2.ppf(0.99, df = 2)
print("Граница в исходной шкале (D²):", round(maha_normal_threshold, 2))
```

	Центр
mean_connections_per_min	119.4806
mean_unique_destinations	12.7916

	mean_connections_per_min	mean_unique_destinations
mean_connections_per_min	320.7583	22.0525
mean_unique_destinations	22.0525	3.8141

```
Граница в исходной шкале (D²): 9.21
```

Шаг 2. Применяем заранее заданное правило к новым данным.

Для объектов тестовой выборки `test` сравниваем значения признака с вычисленной границей.

In [58]:
```
# Для новых точек используем те же центр и ковариацию.

# Вычисляем отклонения тестовых точек от центра (вектора средних значений)
# Результат — матрица разностей: каждая строка — отклонение одной точки
# по всем признакам из JOINT_FEATURES.
maha_differences_test_normal = test[JOINT_FEATURES].to_numpy() - maha_center_nor

# Решаем систему линейных уравнений: C * X = D^T, где C — ковариационная матрица
# (maha_covariance_normal), а D^T — транспонированная матрица отклонений.
# Это шаг нкжен для эффективного вычисления квадратичной формы
# в формуле расстояния Махаланобиса (избегаем явного обращения матрицы, использу
maha_solved_test_normal = np.linalg.solve(maha_covariance_normal, maha_differenc

# Считаем квадраты расстояний Махаланобиса для каждой тестовой точки.
# Формула: d^2 = (x - μ)^T * Σ^(-1) * (x - μ). Здесь это реализовано через поэле
# и суммирование по признакам: (x - μ) умножается на результат решения с Σ, зате
maha_normal_scores_test = np.sum(maha_differences_test_normal.T * maha_solved_te

# Формируем булев массив тревог: точка считается аномальной, если её расстояние
# превышает заданный порог (обычно подбирают по χ²-распределению для заданного у
maha_normal_alerts_test = maha_normal_scores_test > maha_normal_threshold
print(f'Количество тревог: {maha_normal_alerts_test.sum()}')
```

```
Количество тревог: 12
```

Шаг 3. Оцениваем качество работы метода.

После получения тревог присоединяем известную истину для подсчёта ошибок.

Целевой сценарий — `joint`: требуется обнаруживать внедрённое атакующее поведение - необычное совместное поведения двух признаков:

- `mean_connections_per_min` — среднее число соединений в минуту;
- `mean_unique_destinations` — среднее число уникальных адресатов.

При штатной работе эти показатели связаны: увеличение количества соединений обычно сопровождается увеличением числа адресатов. Сценарий `joint` моделирует нарушение этой привычной связи.

Качество детекции будем оценивать с помощью следующих метрик:

- количество верно обнаруженных атак;
- количество ложных срабатываний;
- количество пропущенных атак.

In [59]:
```
maha_normal_target_test = truth.loc[test.index, "attack_joint"].to_numpy()

maha_normal_found = int((maha_normal_alerts_test & maha_normal_target_test).sum(
maha_normal_false = int((maha_normal_alerts_test & ~maha_normal_target_test).sum
maha_normal_missed = int((~maha_normal_alerts_test & maha_normal_target_test).su
```

```
maha_normal_report = pd.Series({"Всего атак": int(maha_normal_target_test.sum())
                                "Срабатываний": int(maha_normal_alerts_test.sum(
                                "Правильно обнаружено": maha_normal_found,
                                "Ложно обнаружено": maha_normal_false,
                                "Пропущено": maha_normal_missed},
                                name = "Махаланобис — на нормальной выборке")
maha_normal_report.to_frame()
```

Out[59]:

	Махаланобис — на нормальной выборке
Всего атак	4
Срабатываний	12
Правильно обнаружено	3
Ложно обнаружено	9
Пропущено	1

Отобразим наблюдения, на которых детектор правильно обнаружил атаки, выдал ложную тревогу и пропустил атаку.

In [60]:

```
maha_normal_events = test[["timestamp", 'mean_connections_per_min', 'mean_unique
maha_normal_events["Целевой сценарий"] = maha_normal_target_test
maha_normal_events["Тревога"] = maha_normal_alerts_test
maha_normal_events["Сведения генератора"] = truth.loc[test.index, "legitimate_no
maha_normal_target_examples = maha_normal_events.loc[maha_normal_target_test]
maha_normal_false_examples = maha_normal_events.loc[maha_normal_alerts_test & ~m
display(pd.concat([maha_normal_target_examples, maha_normal_false_examples]).sor
```

	timestamp	mean_connections_per_min	mean_unique_destinations	Целевой сценарий	Треве
1441	2026-03-02 01:00:00	105.8620	16.6860	False	T
1456	2026-03-02 16:00:00	148.0160	9.9680	False	T
1461	2026-03-02 21:00:00	67.0980	11.6160	False	T
1505	2026-03-04 17:00:00	140.0000	9.0000	True	T
1527	2026-03-05 15:00:00	60.5690	10.5130	False	T
1626	2026-03-09 18:00:00	183.6240	16.6400	False	T
1680	2026-03-12 00:00:00	145.0000	10.0000	True	T
1791	2026-03-16 15:00:00	91.1000	16.0990	False	T
1860	2026-03-19 12:00:00	100.0000	17.0000	True	T
1896	2026-03-21 00:00:00	91.9870	5.9700	False	T
2008	2026-03-25 16:00:00	189.0010	18.8330	False	T
2070	2026-03-28 06:00:00	130.0000	10.7000	True	Fa
2144	2026-03-31 08:00:00	67.2350	6.1810	False	T

Визуализация результатов работы метода.

In [61]:

```
# @title
from matplotlib.patches import Ellipse

# 1. Берём параметры уже обученной модели.
# По проверочным данным модель не переобучаем.
plot_center = maha_center_normal
plot_covariance = maha_covariance_normal
plot_threshold = maha_normal_threshold  # Граница для КВАДРАТА расстояния.

features = ["mean_connections_per_min", "mean_unique_destinations"]

# Каждая строка — два признака одного проверочного часа.
points = test[features].to_numpy(dtype = float)

# 2. Рассчитываем квадрат расстояния Махаланобиса.
deviations = points - plot_center

# Решаем систему вместо явного вычисления обратной матрицы.
solved = np.linalg.solve(plot_covariance, deviations.T)
squared_distances = np.sum(deviations.T * solved, axis=0)

# Тревога возникает только при строгом превышении границы.
alerts = squared_distances > plot_threshold

print("Проверочных наблюдений:", len(points))
print("Выявлено тревог:", int(alerts.sum()))

# 3. Определяем форму и ориентацию эллипса.
# Собственные векторы задают направления его осей,
# собственные значения — разброс вдоль этих направлений.
eigenvalues, eigenvectors = np.linalg.eigh(plot_covariance)

# Сначала располагаем направление с наибольшим разбросом.
order = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[order]
eigenvectors = eigenvectors[:, order]

# Полуоси эллипса: sqrt(порог D² * собственное значение).
semi_axes = np.sqrt(plot_threshold * eigenvalues)

# Угол большой оси относительно горизонтальной оси, в градусах.
angle = np.degrees(np.arctan2(eigenvectors[1, 0], eigenvectors[0, 0]))

# 4. Строим диаграмму.
fig, ax = plt.subplots(figsize=(8, 5))

ax.scatter(points[~alerts, 0], points[~alerts, 1],
           color = "steelblue", s = 25, alpha = 0.5,
           label=f"Без тревоги: {int((~alerts).sum())}")

ax.scatter(points[alerts, 0], points[alerts, 1],
           color = "crimson", marker = "x", s = 70,
           linewidths = 1.8, label = f"Тревоги: {int(alerts.sum())}", zorder = 3

# Ellipse принимает полные длины осей, поэтому умножаем полуоси на 2.
boundary = Ellipse(xy = plot_center,
                   width=2 * semi_axes[0],
                   height = 2 * semi_axes[1],
                   angle = angle,
```

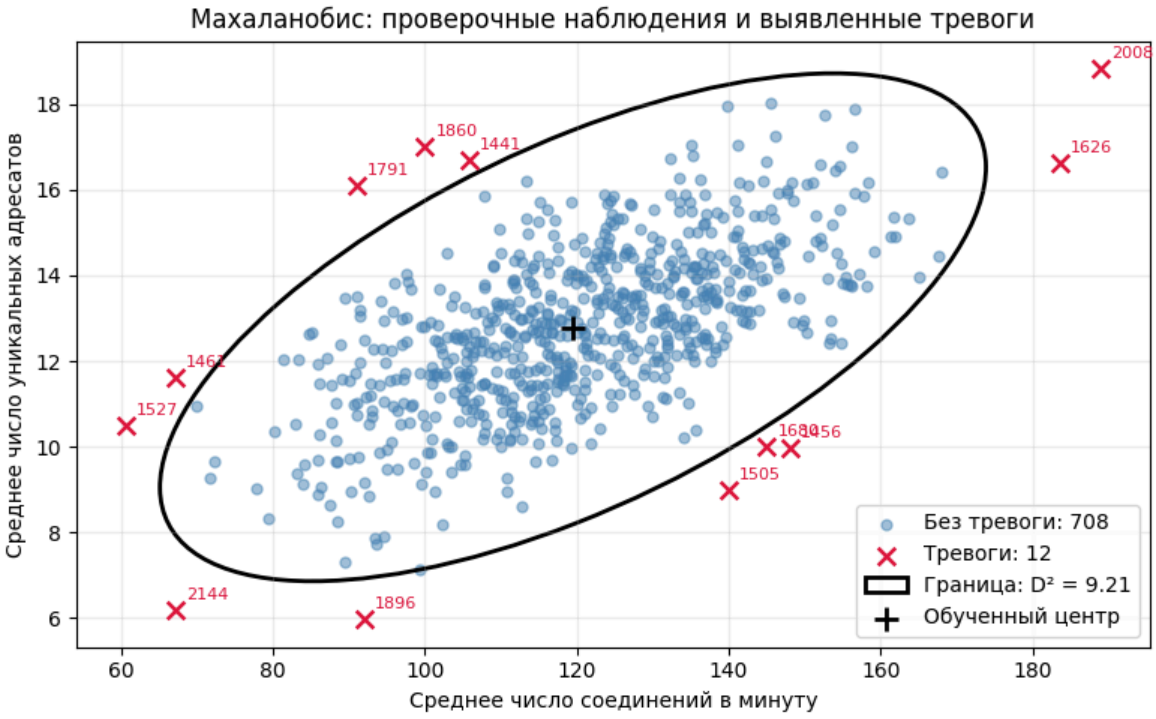
```
                    fill = False,
                    edgecolor = "black",
                    linewidth = 2,
                    label=f"Граница: D² = {plot_threshold:.2f}")
ax.add_patch(boundary)

ax.scatter(plot_center[0], plot_center[1],
           color = "black",  marker = "+",
           s = 140, linewidths = 2, label = "Обученный центр", zorder = 4)

# Подписываем тревоги индексами строк для поиска в таблице test.
for row_index, point in zip(test.index[alerts], points[alerts]):
    ax.annotate(str(row_index),
                xy = (point[0], point[1]),
                xytext = (5, 5),
                textcoords = "offset points",
                fontsize = 8,
                color = "crimson")

ax.set_xlabel("Среднее число соединений в минуту")
ax.set_ylabel("Среднее число уникальных адресатов")
ax.set_title("Махаланобис: проверочные наблюдения и выявленные тревоги")
ax.grid(alpha = 0.25)
ax.legend()
plt.tight_layout()
```

```
Проверочных наблюдений: 720
Выявлено тревог: 12
```



Расстояние Махаланобис в режиме обучения на неразмеченной выборке

Шаг 1. Оцениваем параметры.

Используем выборку `train`.

In [62]:
```
JOINT_FEATURES = ["mean_connections_per_min", "mean_unique_destinations"]
maha_values_unlabelled = train[JOINT_FEATURES].to_numpy(dtype = float)
maha_center_unlabelled = maha_values_unlabelled.mean(axis = 0)
maha_covariance_unlabelled = np.cov(maha_values_unlabelled, rowvar = False, ddof

display(pd.Series(maha_center_unlabelled, index=JOINT_FEATURES, name = "Центр").
display(pd.DataFrame(maha_covariance_unlabelled, index = JOINT_FEATURES, columns

maha_unlabelled_threshold = stats.chi2.ppf(0.99, df = 2)
print("Граница в исходной шкале (D²):", round(maha_unlabelled_threshold, 2))
```

	Центр
mean_connections_per_min	119.4268
mean_unique_destinations	13.0122

	mean_connections_per_min	mean_unique_destinations
mean_connections_per_min	324.8465	25.3588
mean_unique_destinations	25.3588	12.7089

```
Граница в исходной шкале (D²): 9.21
```

Шаг 2. Применяем заранее заданное правило к новым данным.

Для объектов тестовой выборки `test` сравниваем значения признака с вычисленной границей.

In [63]:
```
# Для новых точек используем те же центр и ковариацию.
maha_differences_test_unlabelled = test[JOINT_FEATURES].to_numpy() - maha_center
maha_solved_test_unlabelled = np.linalg.solve(maha_covariance_unlabelled, maha_d

maha_unlabelled_scores_test = np.sum(maha_differences_test_unlabelled.T * maha_s
maha_unlabelled_alerts_test = maha_unlabelled_scores_test > maha_unlabelled_thre
print(f'Количество тревог: {maha_unlabelled_alerts_test.sum()}')
```

```
Количество тревог: 3
```

Шаг 3. Оцениваем качество работы метода.

После получения тревог присоединяем известную истину для подсчёта ошибок.

Целевой сценарий — `joint` : требуется обнаруживать внедрённое атакующее поведение - необычное совместное поведения двух признаков:

- `mean_connections_per_min` — среднее число соединений в минуту;
- `mean_unique_destinations` — среднее число уникальных адресатов.

Качество детекции будем оценивать с помощью следующих метрик:

- количество верно обнаруженных атак;
- количество ложных срабатываний;
- количество пропущенных атак.

In [64]:
```
maha_unlabelled_target_test = truth.loc[test.index, "attack_joint"].to_numpy()

# Каждое выражение сопоставляет тревогу и истину одного и того же часа.
maha_unlabelled_found = int((maha_unlabelled_alerts_test & maha_unlabelled_targe
maha_unlabelled_false = int((maha_unlabelled_alerts_test & ~maha_unlabelled_targ
maha_unlabelled_missed = int((~maha_unlabelled_alerts_test & maha_unlabelled_tar

maha_unlabelled_report = pd.Series({"Всего атак": int(maha_unlabelled_target_tes
                                    "Срабатываний": int(maha_unlabelled_alerts_t
                                    "Правильно обнаружено": maha_unlabelled_foun
                                    "Ложно обнаружено": maha_unlabelled_false,
                                    "Пропущено": maha_unlabelled_missed},
                                    name = "Махаланобис — на неразмеченной выбор
maha_unlabelled_report.to_frame()
```

Out[64]:

	Махаланобис — на неразмеченной выборке
Всего атак	4
Срабатываний	3
Правильно обнаружено	0
Ложно обнаружено	3
Пропущено	4

Отобразим наблюдения, на которых детектор правильно обнаружил атаки, выдал ложную тревогу и пропустил атаку.

In [65]:
```
maha_unlabelled_events = test[["timestamp", 'mean_connections_per_min', 'mean_un
maha_unlabelled_events["Целевой сценарий"] = maha_unlabelled_target_test
maha_unlabelled_events["Тревога"] = maha_unlabelled_alerts_test
maha_unlabelled_events["Сведения генератора"] = truth.loc[test.index, "legitimat
maha_unlabelled_target_examples = maha_unlabelled_events.loc[maha_unlabelled_tar
maha_unlabelled_false_examples = maha_unlabelled_events.loc[maha_unlabelled_aler
display(pd.concat([maha_unlabelled_target_examples, maha_unlabelled_false_exampl
```

	timestamp	mean_connections_per_min	mean_unique_destinations	Целевой сценарий	Треве
1505	2026-03-04 17:00:00	140.0000	9.0000	True	Fa
1527	2026-03-05 15:00:00	60.5690	10.5130	False	T
1626	2026-03-09 18:00:00	183.6240	16.6400	False	T
1680	2026-03-12 00:00:00	145.0000	10.0000	True	Fa
1860	2026-03-19 12:00:00	100.0000	17.0000	True	Fa
2008	2026-03-25 16:00:00	189.0010	18.8330	False	T
2070	2026-03-28 06:00:00	130.0000	10.7000	True	Fa

Расстояние Махаланобиса: сопоставление двух режимов

В обоих режимах использовано одно заранее заданное правило. Различия возникают из-за состава истории и оценённых по ней параметров. Сначала сопоставим параметры, затем фактические решения.

In [66]:
```
maha_comparison = pd.concat([maha_normal_report, maha_unlabelled_report], axis =
display(maha_comparison)
```

	Всего атак	Срабатываний	Правильно обнаружено	Ложно обнаружено	Пропущено
Махаланобис — на нормальной выборке	4	12	3	9	1
Махаланобис — на неразмеченной выборке	4	3	0	3	4

Расстояние Махаланобиса демонстрирует кардинально разную эффективность в зависимости от того, на какой выборке оценены центр и ковариация:

- На нормальной выборке метод чувствителен — находит 3 из 4 атак, но при этом генерирует много ложных срабатываний. Нагрузка на SOC высокая.
- На неразмеченной выборке порог становится почти бесполезным для детекции атак — 0 обнаруженных атак, при этом всё равно есть 3 ложных срабатывания.

Расстояние Махаланобиса эффективно для выявления аномалий в многомерных признаках, когда есть чёткая нормальная корреляционная структура. Однако метод крайне чувствителен к составу обучающей выборки: при наличии аномалий в данных параметры центра и ковариации смещаются, что приводит к потере способности обнаруживать атаки. Это делает метод пригодным только при наличии гарантированно чистой нормальной выборки либо в комбинации с робастными методами.

Ограничения метода.

1. Классическое расстояние предполагает приблизительно многомерное нормальное распределение.
2. Среднее и ковариационная матрица чувствительны к выбросам.
3. При сильной корреляции признаков ковариационная матрица может оказаться плохо обратимой.
4. Число нормальных наблюдений должно заметно превышать число признаков.
5. Метод обнаруживает необычное сочетание, но не объясняет, почему оно возникло.
6. При большом количестве признаков необходимы отбор признаков, регуляризация или робастная оценка ковариации.

5.7. Робастная ковариация MCD

Классическая ковариационная матрица и среднее очень чувствительны к выбросам: даже несколько аномальных наблюдений могут радикально изменить оценку.

Для загрязнённых данных можно применить Minimum Covariance Determinant (MCD) - это робастный метод оценки многомерного центра и разброса данных. Он особенно полезен в задачах кибербезопасности, где выбросы - это не шум, а потенциальные атаки, и классические оценки (среднее, ковариация) сильно искажаются из-за них.

В этом методе вместо использования всех объектов на равных основаниях ищется компактная центральная группа, меньше зависящая от экстремальных значений.

Алгоритм метода.

Пусть у нас n наблюдений. Мы хотим получить устойчивые оценки среднего и ковариации, не испорченные выбросами.

Шаг 1. Задать размер подвыборки h.

Выбираем, сколько точек должно считаться «нормальными». Обычно берут:

$$h = \lceil \alpha \cdot n \rceil, \quad \text{где } \alpha \in [0.5, 0.75].$$

Шаг 2. Внешний цикл: перебор случайных стартов

1. Случайно выбрать h объектов без повторений.
2. Вычислить начальные оценки:
   - $\mu = \text{mean}(X_{\text{start}})$;
   - $\Sigma = \text{cov}(X_{\text{start}})$.
3. **Внутренний цикл: итерации до сходимости.**

   3.1. Вычислить расстояния Махаланобиса для всех n объектов:
   $d_i^2 = (x_i - \mu)^T \Sigma^{-1} (x_i - \mu)$. Это даёт вектор d^2 длины n.

   3.2. Выбрать h точек с наименьшими расстояниями. Отсортировать d^2 по возрастанию, взять индексы первых h объектов.

   3.3. Пересчитать μ и Σ по этим h точкам:
   - $\mu_{\text{new}} = \text{mean}(X_h)$;
   - $\Sigma_{\text{new}} = \text{cov}(X_h)$.

   3.4. Проверить сходимость. Если (μ, Σ) почти не изменились, прервать внутренний цикл. Иначе присвоить $\mu \leftarrow \mu_{\text{new}}$, $\Sigma \leftarrow \Sigma_{\text{new}}$ и повторить.
4. **Оценить качество этого старта**
   После сходимости вычислить определитель ковариционной матрицы:
   $\det = \det(\Sigma)$. Определитель характеризует многомерный масштаб разброса: для эллиптической области меньший определитель соответствует меньшему объёму эллипса (меньшему разбросу).

   Если $\det > 0$ и $\det < \text{best_det}$, обновить:
   - `best_det = det`;
   - `best_mu = mu`;
   - `best_Sigma = Sigma`.

Шаг 3. Финальная робастная оценка.

На выбранных объектах вычислить итоговые робастные параметры:
$\hat{\mu}_{\text{MCD}} = \text{best_mu}$, $\hat{\Sigma}_{\text{MCD}} = \text{best_Sigma}$.

Шаг 4. Детекция аномалий.

1. Для новых объектов посчитать робастные расстояния Махаланобиса:

$$RD_i = \sqrt{(x_i - \hat{\mu}_{\text{MCD}})^\top \hat{\Sigma}_{\text{MCD}}^{1} (x_i - \hat{\mu}_{\text{MCD}})}.$$

2. Сравнивнить с порогом: если, $RD_i > \sqrt{\chi^2_{p,\alpha}}$, точка считается аномалией (где p - число признаков, α - уровень значимости).

Описанный алгоритм робастной ковариации MCD реализован в классе `MinCovDet` библиотеки `sklearn`.

Ключевой параметр `support_fraction`, который задаёт долю объектов, которые считаются «нормальными».

Основные атрибуты:

- `location_`: возвращает робастный центр (mean);
- `covariance_`: возвращает робастную ковариационную матрицу;
- `mahalanobis`: возвращает робастные расстояния Махаланобиса (квадрат не берётся);
- `raw_support_`: возвращает булеву маску (массив длины n), которая показывает, какие объекты вошли подмножество из h объектов.

Робастная ковариация MCD в режиме обучения только на норме

Шаг 1. Оцениваем параметры.

Используем выборку `normal_train`.

In [67]:
```
JOINT_FEATURES
```

Out[67]:
```
['mean_connections_per_min', 'mean_unique_destinations']
```

In [68]:
```
mcd_estimator_normal = MinCovDet(support_fraction = 0.75, random_state = 42)
mcd_estimator_normal.fit(normal_train[JOINT_FEATURES].to_numpy())
mcd_center_normal = mcd_estimator_normal.location_
mcd_covariance_normal = mcd_estimator_normal.covariance_

print("Число точек исходной опоры:", int(mcd_estimator_normal.raw_support_.sum()
display(pd.Series(mcd_center_normal, index=JOINT_FEATURES, name="Робастный центр
display(pd.DataFrame(mcd_covariance_normal, index=JOINT_FEATURES, columns=JOINT_

mcd_normal_threshold = stats.chi2.ppf(0.99, df = 2)
print("Граница в исходной шкале (D²):", round(mcd_normal_threshold, 2))
```

```
Число точек исходной опоры: 501
```

	Робастный центр
mean_connections_per_min	118.9479
mean_unique_destinations	12.7190

	mean_connections_per_min	mean_unique_destinations
mean_connections_per_min	273.2057	17.5151
mean_unique_destinations	17.5151	3.2615

```
Граница в исходной шкале (D²): 9.21
```

Шаг 2. Применяем заранее заданное правило к новым данным.

Для объектов тестовой выборки `test` сравниваем значения признака с вычисленной границей.

In [69]:
```
mcd_normal_scores_test = mcd_estimator_normal.mahalanobis(test[JOINT_FEATURES].t
mcd_normal_alerts_test = mcd_normal_scores_test > mcd_normal_threshold
print(f'Количество тревог: {mcd_normal_alerts_test.sum()}')
```

```
Количество тревог: 15
```

Шаг 3. Оцениваем качество работы метода.

После получения тревог присоединяем известную истину для подсчёта ошибок.

Целевой сценарий — `joint`: требуется обнаруживать внедрённое атакующее поведение - необычное совместное поведения двух признаков:

- `mean_connections_per_min` — среднее число соединений в минуту;
- `mean_unique_destinations` — среднее число уникальных адресатов.

Качество детекции будем оценивать с помощью следующих метрик:

- количество верно обнаруженных атак;
- количество ложных срабатываний;
- количество пропущенных атак.

In [70]:
```
mcd_normal_target_test = truth.loc[test.index, "attack_joint"].to_numpy()

mcd_normal_found = int((mcd_normal_alerts_test & mcd_normal_target_test).sum())
mcd_normal_false = int((mcd_normal_alerts_test & ~mcd_normal_target_test).sum())
mcd_normal_missed = int((~mcd_normal_alerts_test & mcd_normal_target_test).sum()

mcd_normal_report = pd.Series({"Всего атак": int(mcd_normal_target_test.sum()),
                               "Срабатываний": int(mcd_normal_alerts_test.sum())
                               "Правильно обнаружено": mcd_normal_found,
                               "Ложно обнаружено": mcd_normal_false,
                               "Пропущено": mcd_normal_missed},
                               name = "Махаланобис: робастная MCD — на нормально
mcd_normal_report.to_frame()
```

Out[70]:

Махаланобис: робастная MCD — на нормальной выборке	
Всего атак	4
Срабатываний	15
Правильно обнаружено	3
Ложно обнаружено	12
Пропущено	1

Отобразим наблюдения, на которых детектор правильно обнаружил атаки, выдал ложную тревогу и пропустил атаку.

In [71]:
```
mcd_normal_events = test[["timestamp", 'mean_connections_per_min', 'mean_unique_
mcd_normal_events["Целевой сценарий"] = mcd_normal_target_test
mcd_normal_events["Тревога"] = mcd_normal_alerts_test
mcd_normal_events["Сведения генератора"] = truth.loc[test.index, "legitimate_not
mcd_normal_target_examples = mcd_normal_events.loc[mcd_normal_target_test]
mcd_normal_false_examples = mcd_normal_events.loc[mcd_normal_alerts_test & ~mcd_
display(pd.concat([mcd_normal_target_examples, mcd_normal_false_examples]).sort_
```

	timestamp	mean_connections_per_min	mean_unique_destinations	Целевой сценарий	Трев
1441	2026-03-02 01:00:00	105.8620	16.6860	False	T
1456	2026-03-02 16:00:00	148.0160	9.9680	False	T
1461	2026-03-02 21:00:00	67.0980	11.6160	False	T
1505	2026-03-04 17:00:00	140.0000	9.0000	True	T
1527	2026-03-05 15:00:00	60.5690	10.5130	False	T

Робастная ковариация MCD в режиме обучения на неразмеченной выборке

Шаг 1. Оцениваем параметры.

Используем смешанную обучающую выборку `train`, которая содержит как нормальные наблюдения, так и наблюдения с атакующими сценариями.

In [72]:
```
mcd_estimator_unlabelled = MinCovDet(support_fraction = 0.75, random_state = 42)
mcd_estimator_unlabelled.fit(train[JOINT_FEATURES].to_numpy())
mcd_center_unlabelled = mcd_estimator_unlabelled.location_
mcd_covariance_unlabelled = mcd_estimator_unlabelled.covariance_

print("Число точек исходной опоры:", int(mcd_estimator_unlabelled.raw_support_.s
display(pd.Series(mcd_center_unlabelled, index=JOINT_FEATURES, name="Робастный ц
display(pd.DataFrame(mcd_covariance_unlabelled, index=JOINT_FEATURES, columns=JO

# Правило задано в разделе 3.4; по результатам проверки его не меняем.
mcd_unlabelled_threshold = stats.chi2.ppf(0.99, df=2)
print("Граница в исходной шкале (D²):", round(mcd_unlabelled_threshold, 2))
```

```
Число точек исходной опоры: 540
```

	Робастный центр
mean_connections_per_min	118.9210
mean_unique_destinations	12.7353

	mean_connections_per_min	mean_unique_destinations
mean_connections_per_min	278.2856	18.3537
mean_unique_destinations	18.3537	3.4759

```
Граница в исходной шкале (D²): 9.21
```

Шаг 2. Применяем заранее заданное правило к новым данным.

Для объектов тестовой выборки `test` сравниваем значения признака с вычисленной границей.

In [73]:
```
mcd_unlabelled_scores_test = mcd_estimator_unlabelled.mahalanobis(test[JOINT_FEA
mcd_unlabelled_alerts_test = mcd_unlabelled_scores_test > mcd_unlabelled_thresho
print(f'Количество тревог: {mcd_unlabelled_alerts_test.sum()}')
```

```
Количество тревог: 15
```

Шаг 3. Оцениваем качество работы метода.

После получения тревог присоединяем известную истину для подсчёта ошибок.

Целевой сценарий — `joint`: требуется обнаруживать внедрённое атакующее поведение - необычное совместное поведения двух признаков:

- `mean_connections_per_min` — среднее число соединений в минуту;
- `mean_unique_destinations` — среднее число уникальных адресатов.

Качество детекции будем оценивать с помощью следующих метрик:

- количество верно обнаруженных атак;
- количество ложных срабатываний;

- количество пропущенных атак.

In [74]:
```
mcd_unlabelled_target_test = truth.loc[test.index, "attack_joint"].to_numpy()

mcd_unlabelled_found = int((mcd_unlabelled_alerts_test & mcd_unlabelled_target_t
mcd_unlabelled_false = int((mcd_unlabelled_alerts_test & ~mcd_unlabelled_target_
mcd_unlabelled_missed = int((~mcd_unlabelled_alerts_test & mcd_unlabelled_target

mcd_unlabelled_report = pd.Series({"Всего атак": int(mcd_unlabelled_target_test.
                                   "Срабатываний": int(mcd_unlabelled_alerts_tes
                                   "Правильно обнаружено": mcd_unlabelled_found,
                                   "Ложно обнаружено": mcd_unlabelled_false,
                                   "Пропущено": mcd_unlabelled_missed},
                                   name = "Махаланобис: робастная MCD — на нераз
mcd_unlabelled_report.to_frame()
```

Out[74]:

	Махаланобис: робастная MCD — на неразмеченной выборке
Всего атак	4
Срабатываний	15
Правильно обнаружено	3
Ложно обнаружено	12
Пропущено	1

In [75]:
```
mcd_unlabelled_events = test[["timestamp", 'mean_connections_per_min', 'mean_uni
mcd_unlabelled_events["Целевой сценарий"] = mcd_unlabelled_target_test
mcd_unlabelled_events["Тревога"] = mcd_unlabelled_alerts_test
mcd_unlabelled_events["Сведения генератора"] = truth.loc[test.index, "legitimate
mcd_unlabelled_target_examples = mcd_unlabelled_events.loc[mcd_unlabelled_target
mcd_unlabelled_false_examples = mcd_unlabelled_events.loc[mcd_unlabelled_alerts_
display(pd.concat([mcd_unlabelled_target_examples, mcd_unlabelled_false_examples
```

	timestamp	mean_connections_per_min	mean_unique_destinations	Целевой сценарий	Треве
1441	2026-03-02 01:00:00	105.8620	16.6860	False	T
1456	2026-03-02 16:00:00	148.0160	9.9680	False	T
1461	2026-03-02 21:00:00	67.0980	11.6160	False	T
1505	2026-03-04 17:00:00	140.0000	9.0000	True	T
1527	2026-03-05 15:00:00	60.5690	10.5130	False	T

Робастная ковариация MCD: сопоставление двух режимов

В обоих режимах использовано одно заранее заданное правило. Различия возникают из-за состава истории и оценённых по ней параметров. Сначала сопоставим параметры, затем фактические решения.

In [76]:
```
mcd_comparison = pd.concat([mcd_normal_report, mcd_unlabelled_report], axis = 1)
mcd_comparison
```

Out[76]:

	Всего атак	Срабатываний	Правильно обнаружено	Ложно обнаружено	Пропущено
Махаланобис: робастная MCD — на нормальной выборке	4	15	3	12	1
Махаланобис: робастная MCD — на неразмеченной выборке	4	15	3	12	1

Совпадение результатов в двух режимов объясняется устойчивостью MCD к загрязнению истории. Метод оценивает центр и ковариацию по относительно компактной основной группе данных, поэтому отдельные сильные отклонения

могут мало изменить форму эллипса. При одинаковом пороге это способно сохранить решения.

Однако совпадение количеств ещё не доказывает совпадения самих тревог. В двух режимах могли быть отмечены разные строки. Чтобы подтвердить устойчивость именно решений, нужно сопоставить индексы срабатываний, центры и ковариационные матрицы.

Робастный метод формирует достаточно много ложных тревог. Робастность означает устойчивость оценки параметров к загрязнению. Она не гарантирует, что все законные наблюдения попадут внутрь эллипса. Редкие штатные сочетания могут находиться далеко от основной группы; кроме того, эллиптическая модель и выбранный порог могут недостаточно точно описывать нормальное поведение.

5.8. Настройка порога аномальности

Рассмотрим технологию подбора порога аномальности на пример метода Z-score для выявления аномалий в признаке `mean_session_min`.

Классический метод Z-score работает по правилу $Z > 3$. Число 3 задавалось до оценивания качества детекции. Теперь покажем другую процедуру: модель уже обучена на `normal_train`, а границу её оценки будем выбирать по отдельному калибровочному интервалу. Это будет **Z-детектор с настроенным порогом**. Если полученное число отличается от 3, называть новое правило «правилом трёх СКО» нельзя.

Среднее `z_mean_normal` и стандартное отклонение `z_std_normal` не переоцениваются. Для каждого калибровочного объекта вычисляем прежнюю оценку: из длительности вычитаем обученное среднее и делим на обученное стандартное отклонение. Изменяется только граница, с которой сравнивают эту оценку.

Цель остаётся прежней: обнаружение атакующих событий - длительный сеанс `attack_long_session`.

Основная идея.

Порог аномальности T выбирается так, чтобы контролировать долю (или число) ложных срабатываний на заведомо нормальной колибровочной выборке - m. Чаще всего порог берут как квантиль распределения вычисленных оценок аномальности. Это не даёт гарантий на будущее, но даёт воспроизводимую настройку и понятный компромисс между чувствительностью и нагрузкой на аналитиков.

Как формально задать порог по квантилю.

Пусть $z_1 \le z_2 \le \cdots \le z_n$ — упорядоченные значения оценок аномальности объектов на нормальной выборке. Тогда порог T_q определяется как q-квантиль

этого набора.

Эмпирически такой порог отделяет верхнюю долю $1 - q$ наблюдений: ожидается примерно $(1 - q) \cdot n$ точек выше порога. Отсюда ориентир для выбора q под целевое число ложных срабатываний m:

$$q \approx 1 - \frac{m}{n}.$$

Это **ориентир**, а не точная гарантия: из-за дискретности выборки, совпадений значений и способа интерполяции фактическое число превышений может отличаться. Поэтому после выбора q порог нужно явно проверить:

$$\text{число превышений} = \sum_{i=1}^{n} [z_i > T_q]$$

и убедиться, что оно соответствует ожиданиям (например, $\leq m$).

Сценарии настройки порога и что можно оценить.

Сценарий 1: есть только нормальная выборка.

- **Что делаем:** считаем z_i на нормальной выборке, выбираем q, считаем порог T_q, проверяем число превышений.
- **Что можем контролировать:** долю/число ложных срабатываний **на этой выборке**.
- **Чего не можем:** оценить число пропусков атак и полноту, потому что атак в выборке нет.
- **Как использовать:** задать $q \approx 1 - m/n$, проверить, что превышений не больше m, и использовать этот порог как воспроизводимую базовую настройку.

В SOC это типично: берём период, где точно не было атак по результатам расследований, считаем по нему порог, чтобы держать нагрузку на аналитиков в пределах m тревог на n сессий.

Сценарий 2: есть и нормальные, и атакующие метки (размеченная выборка).

- **Что делаем:**
  1. На нормальной части считаем q-квантиль, получаем порог T_q.
  2. Применяем порог ко всей размеченной выборке.
  3. Считаем обе ошибки: ложные срабатывания и пропуски.
- **Что можно оптимизировать:** перебрать несколько q (например, 0.95, 0.97, 0.99, 0.995), для каждого посчитать ощибки, выбрать компромисс под задачу (например, не более 5 ложных срабатываний и максимум обнаруженных атак).
- **Альтернатива:** считать квантиль не по норме, а по всей смеси. Тогда доля $1 - q$ относится к смеси, и она не равна доле ложных срабатываний. Такой подход полезен, если цель — ограничить общий объём тревог, а не именно ложные.

Для SOC чаще используют квантиль по норме: это даёт понятный контроль над ложными срабатываниями и позволяет явно видеть, сколько атак пропускается при таком пороге.

Сценарий 3: нет никаких меток (полностью неразмеченная выборка).

- **Что делаем:** берём наблюдаемые оценки аномальности z_i^2, выбираем q, порог $T_q = \text{quantile}(z, q)$, все точки выше порога помечаем как аномалии.
- **Что можем контролировать:** верхнюю долю тревог в наблюдаемом потоке (например, верхние 0.5% сессий).
- **Чего не можем:** сказать, какие тревоги ложные, а какие — настоящие атаки; не можем оценить число пропусков.
- **Как использовать:** для приоритизации разбора: «возьмём топ-k самых аномальных объектов и вручную разберём их.

Реализуем описанную технологию.

Шаг 1. Рассчитываем оценки аномальности объектов колибровочной выборкии выделяем подтверждённую норму.

Параметры берём из ранее обученной Z-модели. Маска нормы нужна только для построения кандидатов. Позже каждый кандидат проверяется на **всём** калибровочном интервале с прежней целевой меткой.

In [77]:
```
# Стандартизируем признак для калибровочной выборки.
# Используем среднее и стандартное отклонение, посчитанные на обучающей «чистой»
tuning_scores_cal = (calibration["mean_session_min"].to_numpy() - z_mean_normal)

# Строим маску для строк калибровочной выборки, которые являются подтверждённой
tuning_normal_mask = ~truth.loc[calibration.index, attack_columns].any(axis=1).t

# Отбираем из стандартизированных оценок только те, что относятся к подтверждённ
tuning_normal_scores = tuning_scores_cal[tuning_normal_mask]

# Получаем целевую переменную для калибровочной выборки.
# Эта целевая переменная понадобится позже для расчёта метрик качества детекции.
tuning_target_cal = truth.loc[calibration.index, "attack_long_session"].to_numpy

print("Всего калибровочных часов:", len(calibration))
print("Подтверждённо нормальных часов:", len(tuning_normal_scores))
print("Целевых длительных сеансов:", int(tuning_target_cal.sum()))
```

```
Всего калибровочных часов: 720
Подтверждённо нормальных часов: 668
Целевых длительных сеансов: 4
```

Шаг 2. Оцениваем метрики качества детекции для различных порогов аномальности.

Формируем список проверяемых порогов аномальности. Для каждого из кандидатов вычисляем квантиль Z-оценок, вычисленных на норме и применяем границу ко всей размеченной калибровочной выборке.

В последнем столбце отдельно считаем превышения на подтверждённой норме — сколько тревог возникло в наблюдениях, где вообще не было внедрённых атак.

«Ложно обнаружено» считается относительно `long_session`: оно включает любые наблюдения с тревогой без этой цели, в том числе наблюдения с другим типом атаки.

In [78]:
```
tuning_levels = [0.95, 0.975, 0.99, 0.995, 0.999]
tuning_rows = []
for level in tuning_levels:
    candidate_threshold = np.quantile(tuning_normal_scores, level)
    candidate_alerts = tuning_scores_cal > candidate_threshold
    tuning_rows.append({"Уровень q": level,
                        "Порог Z": candidate_threshold,
                        "Всего атак": int(tuning_target_cal.sum()),
                        "Срабатываний": int(candidate_alerts.sum()),
                        "Правильно обнаружено": int((candidate_alerts & tuning_t
                        "Ложно обнаружено": int((candidate_alerts & ~tuning_targ
                        "Пропущено": int((~candidate_alerts & tuning_target_cal)
tuning_table = pd.DataFrame(tuning_rows)
tuning_table
```

Out[78]:

	Уровень q	Порог Z	Всего атак	Срабатываний	Правильно обнаружено	Ложно обнаружено	Пропущено
0	0.9500	1.6019	4	40	4	36	0
1	0.9750	1.8299	4	23	4	19	0
2	0.9900	2.1796	4	10	3	7	1
3	0.9950	2.2585	4	7	3	4	1
4	0.9990	3.7808	4	3	2	1	2

Шаг 3. Выбираем порог аномальности по заданному условию.

Будем использовать условие: **не более пяти ложных обнаружений на полном калибровочном интервале**.

Среди допустимых кандидатов предпочитаем обеспечивающих меньше пропусков атак, затем - меньше ложных срабатываний, при полном равенстве — более строгий порог. Если все кандидаты дают больше пяти ложных тревог, то подходящего варианта нет. Тогда до итоговой проверки нужно попробовать другие пороги, изменить модель либо обоснованно пересмотреть допустимое число ошибок. Выполнение условия на калибровке не гарантирует такого же результата на будущих данных.

В соответсвии с заданным условием выбран кваниль $q = 0.995$ и порог $Z = 2.2585$. На калибровке правильно обнаружено 3, ложно обнаружено 4, пропущено 1. Он удовлетворяет ограничению и заданному порядку предпочтений среди кандидатов.

In [79]:
```
tuning_chosen_level = 0.995
tuning_threshold = np.quantile(tuning_normal_scores, tuning_chosen_level)
```

```
tuning_selected_row = tuning_table.loc[tuning_table["Уровень q"].eq(tuning_chose
display(tuning_selected_row)
assert len(tuning_selected_row) == 1
assert tuning_selected_row["Ложно обнаружено"].le(5).all()
print("Зафиксированный порог Z:", round(tuning_threshold, 2))
print("Соответствующая граница длительности, мин:", round(z_mean_normal + tuning
```

	Уровень q	Порог Z	Всего атак	Срабатываний	Правильно обнаружено	Ложно обнаружено	Пропущено
3	0.9950	2.2585	4	7	3	4	1

```
Зафиксированный порог Z: 2.26
Соответствующая граница длительности, мин: 49.12
```

Шаг 4. Проверяем найденный порог аномальности на тестовой выборке.

Ни модель, ни уровень квантиля, ни граница теперь не меняются.

Сравним качество детекции при использовании найденного порога с качеством при фиксированном пороге $Z > 3$.

In [80]:
```
tuning_scores_test = (test["mean_session_min"].to_numpy() - z_mean_normal) / z_s
tuning_alerts_test = tuning_scores_test > tuning_threshold
tuning_target_test = truth.loc[test.index, "attack_long_session"].to_numpy()
tuning_report = pd.Series({"Всего атак": int(tuning_target_test.sum()),
                           "Срабатываний": int(tuning_alerts_test.sum()),
                           "Правильно обнаружено": int((tuning_alerts_test & tun
                           "Ложно обнаружено": int((tuning_alerts_test & ~tuning
                           "Пропущено": int((~tuning_alerts_test & tuning_target
                           name = "Z-score — порог по нормальной калибровке")
display(pd.concat([z_normal_report, tuning_report], axis = 1).T)
```

	Всего атак	Срабатываний	Правильно обнаружено	Ложно обнаружено	Пропущено
Z-score — на нормальной выборке	4	5	3	2	1
Z-score — порог по нормальной калибровке	4	12	3	9	1

Разбор результата. После настройки на проверке правильно обнаружено 3, ложно обнаружено 9, пропущено 1. Для исходного правила Z > 3: правильно обнаружено 3, ложно обнаружено 2, пропущено 1; всего тревог 5. Настроенная граница длительности равна 49.118 мин. Сравнение объясняет влияние границы; оно не доказывает универсального улучшения.

Ограничение на калибровке не гарантирует такого же числа ложных тревог в будущем. Другие хвостовые наблюдения, изменение нагрузки и непредставительность нормы могут изменить результат. Сам по себе квантиль не устраняет эти причины.

В этом примере форма оценки обучена на нормальной истории, а выбор уровня использует размеченную калибровку. Это настройка с помощью меток. Если оставить только подтверждённую норму и задать уровень заранее, процедура будет другой: число пропусков не сможет участвовать в выборе.

5.9. Контрольные карты Шухарта

Методология применения

Контрольные карты применяются для анализа поведения процесса во времени и основаны на идее разделения изменчивости процесса на две категории:

- общие (случайные) причины — естественная вариативность, которая всегда присутствует;
- особые (специальные) причины — конкретные, идентифицируемые факторы, которые нарушают стабильность.

Карты Шухарта — это графики, где по оси X откладывают номер подгруппы (или время), а по оси Y — значение некоторой статистики (среднее, размах, доля и т. п.) для этой подгруппы.

На карту наносят три линии:

- центральная линия (CL) — среднее значение статистики, рассчитанное по выборочным данным;
- верхняя контрольная граница (UCL) и нижняя контрольная граница (LCL)

Если наблюдение выходит за контрольную границу, считается, что процесс мог перейти в необычное состояние.

Как правило, строятся парные контрольные карты Шухарта: $\bar{X}$-карта и S-карта, потому что они отвечают на разные вопросы о стабильности процесса.

$\bar{X}$-карта (карта средних) показывает, сдвигается ли центр процесса (меняется ли среднее значение). По оси Y — среднее по подгруппе. Если среднее превысило границы - это сигнал о систематическом изменении (например, начался DDoS или изменился профиль трафика).

S-карта (карта стандартных отклонений) показывает, насколько стабилен разброс внутри подгруппы: если разброс резко вырос - процесс стал менее предсказуемым (скачки, сканирование, латеральное движение и т. п.).

Вычисление пороговых значений аномальности.

Будем использовать следующие обозначения:

- n — количество наблюдений в одной подгруппе;
- m — количество полных подгрупп;

- $\bar{x}_i$ — среднее подгруппы i;
- s_i — стандартное отклонение внутри подгруппы i.

По нормальной выборке рассчитываем:

$$\bar{\bar{x}} = \frac{1}{m}\sum_{i=1}^{m}\bar{x}_i, \qquad \bar{s} = \frac{1}{m}\sum_{i=1}^{m}s_i.$$

Здесь $\bar{\bar{x}}$ — средний уровень процесса, а $\bar{s}$ — средний разброс процесса.

Чтобы получить несмещённую оценку средего разброса процесса, используют поправочный коэффициент c_4:

$$\widehat{\sigma} = \frac{\bar{s}}{c_4}, \quad \text{где } c_4 = \sqrt{\frac{2}{n-1}}\cdot\frac{\Gamma(n/2)}{\Gamma((n-1)/2)}.$$

Для X̄-карты:

- центральная линия:

$$CL_{\bar{X}} = \bar{\bar{x}},$$

- нижняя и верхняя границы:

$$LCL_{\bar{X}} = \bar{\bar{x}} - 3\frac{\widehat{\sigma}}{\sqrt{n}}, \qquad UCL_{\bar{X}} = \bar{\bar{x}} + 3\frac{\widehat{\sigma}}{\sqrt{n}}.$$

Для S-карты:

- центральная линия:

$$CL_S = \bar{s},$$

- нижняя и верхняя границы:

$$LCL_S = \max\left(0,\ \bar{s} - 3\widehat{\sigma}\sqrt{1-c_4^2}\right), \qquad UCL_S = \bar{s} + 3\widehat{\sigma}\sqrt{1-c_4^2}.$$

Рассмотрим технологию применения контрольных карт на примере признака `mean_requests_per_min` — среднее число запросов в минуту, рассчитанное за каждый час.

В нашем наборе данных внедрен атакующий сценарий `shift` — устойчивое повышение среденего уровня числа запросов.

Предположим, обычно нагрузка колеблется около 100 запросов в минуту. Затем несколько часов подряд она держится около 120:

Час	Среднее число запросов в минуту	Что происходит
10:00–11:00	98	Обычный уровень
11:00–12:00	103	Обычный уровень
12:00–13:00	121	Начало повышения

Час	Среднее число запросов в минуту	Что происходит
13:00–14:00	124	Повышение продолжается
14:00–15:00	119	Повышение продолжается
15:00–16:00	101	Возврат к обычному уровню

Здесь важен не один сильный скачок, а несколько последовательных повышенных значений. Каждое из них по отдельности может не превышать границу тревоги, однако вся последовательность указывает на изменение поведения. Цель детектора — заметить этот устойчивый рост.

В учебной разметке `attack_shift = True` отмечает часы, в которых генератор внедрил такое повышение. В примере выше — это три часовых наблюдения, относящиеся к одному периоду повышенной нагрузки.

Каждая подгруппа будет объедять шесть последовательных наблюдений (часов). В результате одна точка на картах будет соответствовать шести часам.

Для обнаружения атак типа `shift`:

- $\bar{X}$-карта — основной инструмент. Ищем превышение верхней границы среднего;
- S-карта — дополнительный инструмент. Она показывает, изменился ли разброс нагрузки между часами.

Контрольные карты Шухарта в режиме обучения только на норме

1. Подготавливаем подгруппы.

Используем выборки `normal_train` и `test`. Подгруппы задаются календарными интервалами: 00:00–06:00, 06:00–12:00 и иак далее. Некоторые часы отсутствуют, поэтому неполные подгруппы исключаем целиком: соединять оставшиеся строки через временной разрыв нельзя.

In [83]:
```
FEATURE = "mean_requests_per_min"
GROUP_HOURS = 6

# Каждая подгруппа содержит шесть последовательных часовых значений.
# mean — среднее внутри подгруппы.
# std — выборочное стандартное отклонение, с делителем n - 1.
# count — количество имеющихся значений.
#
# label="right": индексом становится конец интервала.
# Например, точка 06:00 описывает часы 00:00–06:00.
train_groups = (normal_train
    .set_index("timestamp")[FEATURE]
    .resample("6h",
              origin="start_day",
              closed="left",
              label="right").agg(["count", "mean", "std"]))
```

```
test_groups = (test
    .set_index("timestamp")[FEATURE]
    .resample("6h",
              origin="start_day",
              closed="left",
              label="right").agg(["count", "mean", "std"]))

# Используем только полные подгруппы одинакового размера.
train_groups = train_groups.loc[train_groups["count"].eq(GROUP_HOURS)].copy()
test_groups = test_groups.loc[test_groups["count"].eq(GROUP_HOURS)].copy()

print("Нормальных обучающих подгрупп:", len(train_groups))
print("Проверочных подгрупп:", len(test_groups))

train_groups.head()
```

```
Нормальных обучающих подгрупп: 96
Проверочных подгрупп: 120
```

Out[83]:

timestamp	count	mean	std
2026-01-01 06:00:00	6	94.7870	13.6748
2026-01-01 12:00:00	6	95.1382	10.9443
2026-01-01 18:00:00	6	103.5742	9.9174
2026-01-02 00:00:00	6	93.3488	14.4388
2026-01-02 06:00:00	6	96.7353	9.9501

Шаг 2. Вычисляем пороговые значения аномальности.

Используем выборку `normal_train`.

In [86]:

```
from scipy.special import gammaln

# Размер подгруппы и количество обучающих подгрупп.
n = GROUP_HOURS
m = len(train_groups)

# Центральные линии.
central_mean = train_groups["mean"].mean()
central_s = train_groups["std"].mean()

# Поправочный коэффициент:
c4 = np.sqrt(2 / (n - 1)) * np.exp(gammaln(n / 2) - gammaln((n - 1) / 2))

# Оценка стандартного отклонения отдельных часовых значений.
sigma_estimate = central_s / c4

# Границы X-карты.
x_center = central_mean
x_scale = sigma_estimate / np.sqrt(n)

x_lower = x_center - 3 * x_scale
x_upper = x_center + 3 * x_scale
```

```
# Границы S-карты.
s_center = central_s
s_scale = sigma_estimate * np.sqrt(1 - c4**2)

s_lower = max(0.0, s_center - 3 * s_scale)
s_upper = s_center + 3 * s_scale

limits = pd.DataFrame({"Карта": ["X-bar", "S"],
                       "Нижняя граница": [x_lower, s_lower],
                       "Центральная линия": [x_center, s_center],
                       "Верхняя граница": [x_upper, s_upper]}).set_index("Карта"

print("Размер подгруппы:", n)
print("Обучающих подгрупп:", m)
print("Поправочный коэффициент c4:", round(c4, 6))
limits
```

```
Размер подгруппы: 6
Обучающих подгрупп: 96
Поправочный коэффициент c4: 0.951533
```

Out[86]:

Карта	Нижняя граница	Центральная линия	Верхняя граница
X-bar	88.6676	99.2737	109.8799
S	0.2502	8.2402	16.2302

Полученные границы фиксируются. При проверке они не пересчитываются по новым наблюдениям.

Шаг 3. Определяем тревоги и строим обе карты.

In [90]:

```
# Выход среднего подгруппы за границы X-bar-карты.
x_alerts = ((test_groups["mean"] < x_lower) | (test_groups["mean"] > x_upper))

# Выход стандартного отклонения за границы S-карты.
s_alerts = ((test_groups["std"] < s_lower) | (test_groups["std"] > s_upper))

fig, axes = plt.subplots(2, 1, figsize=(10, 8), sharex = True)

# X-карта.
axes[0].plot(test_groups.index, test_groups["mean"], marker = "o",
             markersize = 3, linewidth = 0.8, color = "steelblue",
             label = "Среднее подгруппы")

axes[0].axhline(x_center, color = "seagreen", label = "Центральная линия")
axes[0].axhline(x_upper, color = "black", linestyle = "--", label = "Контрольные
axes[0].axhline(x_lower, color = "black", linestyle = "--")

axes[0].scatter(test_groups.index[x_alerts], test_groups.loc[x_alerts, "mean"],
                color = "crimson", s = 55, zorder = 3, label="Тревоги")

axes[0].set_title("X-карта: средний уровень нагрузки")
axes[0].set_ylabel("Запросов в минуту")
axes[0].grid(alpha = 0.25)
axes[0].legend()
```

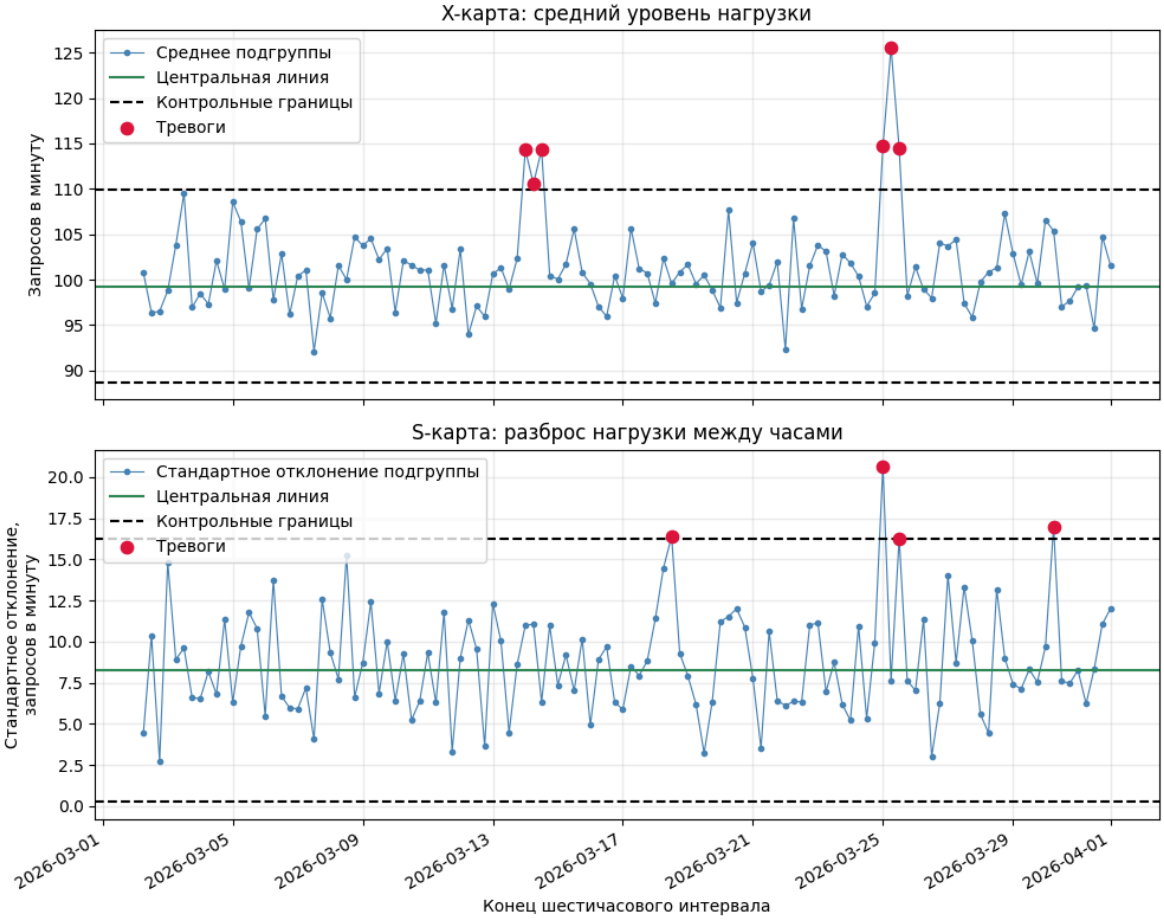
```
# S-карта.
axes[1].plot(test_groups.index, test_groups["std"], marker = "o",
             markersize = 3, linewidth = 0.8, color = "steelblue",
             label = "Стандартное отклонение подгруппы")

axes[1].axhline(s_center, color = "seagreen", label = "Центральная линия")
axes[1].axhline(s_upper, color = "black", linestyle = "--", label = "Контрольные
axes[1].axhline(s_lower, color = "black", linestyle = "--")

axes[1].scatter(test_groups.index[s_alerts], test_groups.loc[s_alerts, "std"],
                color = "crimson", s = 55, zorder = 3, label = "Тревоги")

axes[1].set_title("S-карта: разброс нагрузки между часами")
axes[1].set_ylabel("Стандартное отклонение,\nзапросов в минуту")
axes[1].set_xlabel("Конец шестичасового интервала")
axes[1].grid(alpha = 0.25)
axes[1].legend()

fig.autofmt_xdate()
plt.tight_layout()
```



Шаг 4. Формирование разметки наличия атаки по подгруппам.

Разметка `attack_shift` задана для отдельных часов, а карты принимают решения по шестичасовым подгруппам. Поэтому сначала нужно согласовать разметку с решениями.

Примем правило: подгруппа содержит целевой сценарий, если хотя бы один из её шести часов имеет метку `attack_shift = True`.

Например:

Часов с `shift` внутри подгруппы	Как оцениваем подгруппу
0	Целевого сценария нет
1	Целевой сценарий есть
4	Целевой сценарий есть
6	Целевой сценарий есть

Таким образом, найденная подгруппа означает, что тревога возникла на интервале, содержащем `shift`. Это не означает обнаружение каждого атакующего часа или точного момента начала повышения. Решение становится доступно после завершения подгруппы.

In [101…

```
# Присоединяем известную разметку к проверочным часам.
shift_hourly = test[["timestamp"]].copy()
shift_hourly["attack_shift"] = truth.loc[test.index, "attack_shift"].to_numpy()

# Группируем разметку точно так же, как значения признака.
# sum считает количество часов с attack_shift = True.
shift_group_truth = (
    shift_hourly
    .set_index("timestamp")["attack_shift"]
    .resample("6h", origin = "start_day", closed = "left", label = "right")
    .agg(["count", "sum"]))

# Берём только подгруппы, для которых ранее рассчитаны среднее и std.
shift_check = test_groups[["mean", "std"]].copy()
shift_check["Часов с shift"] = (shift_group_truth["sum"].reindex(shift_check.ind

# Подгруппа целевая, если содержит хотя бы один час с shift.
shift_check["Есть shift"] = shift_check["Часов с shift"] > 0
shift_target = shift_check["Есть shift"]

print("Проверочных подгрупп:", len(shift_check))
print("Часов с shift:", int(shift_check["Часов с shift"].sum()))
print("Подгрупп, содержащих shift:", int(shift_target.sum()))
```

```
Проверочных подгрупп: 120
Часов с shift: 36
Подгрупп, содержащих shift: 8
```

Шаг 4. Оценивание качества детекции.

Цель — обнаружить сценарий `shift`: внедрённое устойчивое повышение среднего уровня нагрузки.

Сравним два заранее заданных правила:

1. Только $\bar{X}$-карта: среднее подгруппы выше верхней границы.
2. $\bar{X}$-карта или S-карта: превышена верхняя граница среднего либо стандартное отклонение вышло за одну из границ S-карты.

Второе правило охватывает больше изменений. Проверим, помогает ли это обнаруживать именно `shift`.

Оценивание только X̄-карты.

Поскольку ищем повышение нагрузки, используем верхнюю границу:

$$\bar{x}_i > UCL_{\bar{X}},$$

где $\bar{x}_i$ — среднее проверочной подгруппы, $UCL_{\bar{X}}$ — верхняя граница, рассчитанная по нормальной истории.

Подсчитываем:

- правильно обнаружено — тревога есть, подгруппа содержит `shift`;
- ложно обнаружено — тревога есть, подгруппа не содержит `shift`;
- пропущено — подгруппа содержит `shift`, тревоги нет.

«Ложно обнаружено» здесь определяется относительно `shift`. Другие атакующие сценарии сами по себе не делают подгруппу целевой для этой проверки.

In [103…

```
# Решение принимается только по среднему и зафиксированной границе.
shift_alerts_x = shift_check["mean"] > x_upper

# После получения решений сопоставляем их с разметкой.
shift_report_x = pd.Series({"Интервалов с shift": int(shift_target.sum()),
                            "Срабатываний": int(shift_alerts_x.sum()),
                            "Правильно обнаружено": int((shift_alerts_x & shift_
                            "Ложно обнаружено": int((shift_alerts_x & ~shift_tar
                            "Пропущено": int((~shift_alerts_x & shift_target).su
                            name = "Качество детекции по X-карте")

shift_report_x.to_frame("Количество")
```

Out[103…

	Количество
Интервалов с shift	8
Срабатываний	6
Правильно обнаружено	6
Ложно обнаружено	0
Пропущено	2

Оценивание совместного использования X̄-карты и S-карты.

Тревогу формируем, если выполнено хотя бы одно условие:

$$\bar{x}_i > UCL_{\bar{X}} \quad \text{или} \quad s_i < LCL_S \quad \text{или} \quad s_i > UCL_S,$$

где s_i — стандартное отклонение подгруппы, LCL_S и UCL_S — границы S-карты.

Используем логическое ИЛИ. Требование одновременного срабатывания обеих карт было бы другим правилом: устойчивое повышение среднего может

происходить без увеличения разброса.

In [104…
```
# S-карта отмечает необычно низкий или высокий разброс.
shift_alerts_s = ((shift_check["std"] < s_lower) | (shift_check["std"] > s_upper

# Достаточно срабатывания хотя бы одной карты.
shift_alerts_xs = shift_alerts_x | shift_alerts_s

# Оцениваем объединённое правило по той же разметке.
shift_report_xs = pd.Series({"Интервалов с shift": int(shift_target.sum()),
                             "Срабатываний": int(shift_alerts_xs.sum()),
                             "Правильно обнаружено": int((shift_alerts_xs & shif
                             "Ложно обнаружено": int((shift_alerts_xs & ~shift_t
                             "Пропущено": int((~shift_alerts_xs & shift_target).
                             name = "X-bar ИЛИ S")
shift_report_xs.to_frame("Количество")
```

Out[104…

	Количество
Интервалов с shift	8
Срабатываний	8
Правильно обнаружено	6
Ложно обнаружено	2
Пропущено	2

Сопоставляем результаты.

In [105…
```
shift_comparison = pd.concat([shift_report_x, shift_report_xs], axis = 1).T
shift_comparison
```

Out[105…

	Интервалов с shift	Срабатываний	Правильно обнаружено	Ложно обнаружено	Пропущено
Качество детекции по X-карте	8	6	6	0	2
X-bar ИЛИ S	8	8	6	2	2

Только $\bar{X}$-карта. Из восьми целевых подгрупп обнаружены шесть. Две пропущены: их средние не превысили верхнюю границу. Ложных тревог не возникло.

Совместное правило. Число найденных целевых подгрупп осталось прежним. S-карта добавила две тревоги на интервалах без `shift`, поэтому ложных обнаружений стало больше.

В этом опыте добавление S-карты не улучшило обнаружение `shift`. Она отреагировала на необычный разброс, однако эти дополнительные срабатывания

не помогли найти два пропущенных целевых интервала. При изменении данных результаты могут отличаться.

6. Практические рекомендации по применению статистических методов

1. Разделять наблюдения на базовый и анализируемый интервалы.

Статистические параметры должны рассчитываться на историческом интервале, который считается нормальным.

2. Строить отдельные профили.

Общий порог для всей организации часто недостаточен. Целесообразно строить отдельные профили:

- для каждого пользователя;
- для каждой роли;
- для каждого устройства;
- для серверов и рабочих станций;
- для рабочего и нерабочего времени;
- для разных дней недели.

3. Не считать каждую статистическую аномалию атакой.

Результат статистического метода следует интерпретировать как: «это событие требует дополнительной проверки».

Для принятия решения необходимо добавить контекст:

- кому принадлежит учётная запись;
- с какого устройства выполнено действие;
- к какому ресурсу обращался пользователь;
- существовала ли заявка на проведение работ;
- какие события происходили до и после аномалии;
- наблюдались ли другие признаки компрометации.

В кибербезопасности статистическая аномалия является основанием для расследования, но не доказательством атаки. Наиболее эффективный подход заключается в сочетании статистических методов, контекстной информации, экспертных правил и последующей проверки аналитиком.

Литература

1. Чио К., Фримэн Д. Машинное обучение и безопасность [Электронный ресурс]:. - Москва: ДМК Пресс, 2020. - 388 с. – Режим доступа:

https://e.lanbook.com/book/131707.
2. Протодьяконов А. В., Пылов П. А., Садовников В. Е. Алгоритмы Data Science и их практическая реализация на Python [Электронный ресурс]:учебное пособие. - Вологда: Инфра-Инженерия, 2022. - 392 с. – Режим доступа: https://e.lanbook.com/book/281189.
3. Криволапов С.Я. Анализ данных. Методы теории вероятностей и математической статистики на языке Python [Электронный ресурс]:Учебное пособие. - Москва: ООО "Научно-издательский центр ИНФРА-М", 2025. - 678 с. – Режим доступа: https://znanium.ru/catalog/document?id=450872.
4. Коротеев М.В. Основы машинного обучения на Python [Электронный ресурс]:Учебник. - Москва: КноРус, 2025. - 431 с. – Режим доступа: https://book.ru/book/957785.
5. Груздев А. В. План, примеры и метрики качества [Электронный ресурс]:. - Москва: ДМК Пресс, 2023. - 814 с. – Режим доступа: https://e.lanbook.com/book/314948.
6. Груздев А. В. Инструменты и валидация [Электронный ресурс]:. - Москва: ДМК Пресс, 2023. - 816 с. – Режим доступа: https://e.lanbook.com/book/314945.